

This is a copy of a conversation between ChatGPT & Anonymous.

Report conversation

I am going to start working with this board B-G431B-ESC1, and I want you to make an outline of how I should address this study my goal is to develop a FOC senseless algorithm for a brushless, and I want to control the motor with serial or CAN input

Here's a practical study outline to reach your goal: **sensorless FOC on the B-G431B-ESC1 + motor control via UART or CAN.**

0) Target definition (1–2 sessions)

- Define what “done” means: speed control only, or speed + torque, or position too.
- Decide your **command interface** first:
 - UART: simplest for early bring-up.
 - CAN: better for robust systems (IDs, priorities, multi-node), and the board supports it. STMicroelectron... +1
- Pick your motor: KV, pole pairs, max current, supply voltage range, expected speed range.

Deliverable: requirements sheet (motor + supply + max current + interface + control modes).

1) Board bring-up and safety (1–2 sessions)

Study and verify the B-G431B-ESC1 hardware capabilities:

- **3-shunt current sensing**, power stage/driver, protections (OCP/OVP), NTC temperature, and available comms (UART/PWM/CAN). STMicroelectron... +1
- Identify connectors/pads for:
 - Motor phases, DC bus, ST-LINK daughterboard, UART pins, CAN pads/transceiver. STMicroelectron... +1
- Bring-up checklist:
 - Power supply with current limit
 - No-load motor test first
 - Fault handling (overcurrent/overtemp), emergency stop behavior

Deliverable: a “power-up checklist” + pin/connector map for your lab notes.

2) Toolchain and ST ecosystem path (fastest route) (2–4 sessions)

Use the ST motor-control ecosystem to get a working baseline quickly:

- Install **X-CUBE-MCSDK** (FOC firmware library + Motor Control Workbench).
STMicroelectron... +1
- Use **Motor Control Workbench** to configure your motor and generate a project.
- Use **STM32 MC Motor Pilot** to monitor variables, tune loops, and validate behavior live. STMicroelectron...

Deliverable: first working closed-loop FOC demo (even if it's ST's reference sensorless).

3) FOC fundamentals on STM32G4 (core study block) (3–6 sessions)

Study and document (side-by-side with the generated code) these modules:

- **Sampling & timing:** PWM center-aligned, ADC triggering, current reconstruction with 3-shunts
- **Clarke/Park** transforms
- **Current controllers** (PI for I_d , I_q) + decoupling terms
- **SVPWM** generation
- **Speed loop** (PI) and optional torque mode
- **State machine:** IDLE → ALIGN → STARTUP → CLOSED_LOOP → FAULT

Deliverable: a "FOC pipeline diagram" mapped to functions/ISR timing.

4) Sensorless strategy (the hard part) (4–10 sessions)

Break it into three sub-problems:

4.1 Rotor angle estimation at medium/high speed

Common approaches you'll see in ST ecosystems:

- Back-EMF observer + PLL / CORDIC-based observer (classic STO)
- Newer high-sensitivity observers (if available in your MCSDK version) Manuals+

What to learn:

- How observer gains relate to $R_s/L_s/K_e$
- Noise sensitivity vs bandwidth
- How angle error shows up as I_d drift / torque ripple

4.2 Start-up from standstill (no back-EMF)

Typical sequence:

- **Rotor alignment** (force a known electrical angle using I_d)
- **Open-loop ramp** (electrical frequency ramp + controlled I_q)

- **Hand-over** to observer when it becomes reliable

Key study items:

- Ramp profiles and current limits
- Criteria for “observer lock” (angle error, PLL convergence, speed threshold)
- Smooth blending (avoid torque steps)

4.3 Low-speed performance (optional “advanced”)

- If you need strong low-speed torque without sensors: study HF injection methods (more complex), or use a hybrid approach (open-loop/observer blending).

Deliverable: documented startup + handover logic with clear thresholds.

5) Motor identification and calibration workflow (2–6 sessions)

To make sensorless FOC robust, you need good parameters:

- Pole pairs
- Phase resistance R_s
- Inductances L_d/L_q (or a usable approximation)
- Back-EMF constant (K_e)

Use Workbench/Profiler where possible, then validate with your own tests. STMicroelectron... +1

Deliverable: a parameter sheet + a repeatable “commissioning procedure”.

6) Command interface: UART first, then CAN (2–6 sessions)

6.1 UART command layer (recommended first)

- Define a small protocol:
 - `SET_MODE` , `SET_SPEED_RPM` , `SET_IQ` , `STOP` , `GET_STATUS` , `GET_FAULTS`
- Implement:
 - RX parsing (DMA + ring buffer)
 - Command timeout (safety)
 - Telemetry streaming at fixed rate (optional)

6.2 CAN command layer (after control is stable)

The board supports CAN and includes the transceiver/pads. STMicroelectron... +1

- Choose a CAN profile:
 - Simple custom: 11-bit IDs with a few message types
 - Or CANopen (more work, more standard)
- Define:
 - Node ID strategy

- Heartbeat/status frames
- Fault frame with latched faults
- Rate limits (don't overload the bus)

Deliverable: a minimal "CAN interface spec" (IDs, payloads, rates).

7) Testing, robustness, and "engineering finish" (ongoing)

- Unit tests (where possible) + hardware-in-loop style checks
- Fault injection:
 - Undervoltage/overvoltage behavior
 - Overtemp
 - Stall / abrupt load steps
- Performance metrics:
 - Startup success rate
 - Torque ripple (qualitative at least)
 - Speed regulation error
 - Current limits respected

Deliverable: test report template + plots from Motor Pilot/telemetry.

Suggested milestone path (so you don't get stuck)

1. Run ST demo / generated project on B-G431B-ESC1 (closed-loop FOC works).
STMicroelectron... +1
2. Understand and annotate the full control pipeline (ISR timing, transforms, PI, SVPWM).
3. Make startup + sensorless handover stable across your operating range.
4. Add UART control + telemetry.
5. Add CAN control + heartbeat/fault frames.

If you want, paste the **motor model (KV, pole pairs, rated current/voltage)** and your intended **DC bus voltage**, and I'll turn this into a week-by-week plan with concrete experiments and expected plots (currents, estimated angle, speed, observer lock indicators).

 Sources

speed control, uart, motor 2852 3100kv rlaarlo, bateria 3s
2200mah

Good — with **speed control + UART + 2852 3100KV + 3S** (≈ 11.1 V nominal, 12.6 V full)

you can set a very concrete path.

Quick numbers (so your limits make sense)

- No-load max speed (rough):
 - nominal: $3100 \times 11.1 \approx 34,410$ rpm
 - fully charged: $3100 \times 12.6 \approx 39,060$ rpm
- Electrical frequency at top speed depends on pole pairs. Many 28xx outrunners are **7 pole pairs (14 poles)**, but verify.
If 7 pp: $39,060 \text{ rpm} \rightarrow 651 \text{ rps} \rightarrow \sim 4.56 \text{ kHz electrical}$.

This tells you: high electrical frequency + sensorless = you must be careful with sampling/PWM and observer bandwidth.

Study / implementation outline (tailored to your setup)

1) Hardware bring-up and safe limits (Session 1–2)

- Use a bench PSU with current limit if possible (start low, e.g., 1–2 A).
- Confirm:
 - Motor phase order and smooth rotation in open loop.
 - Current sensing is reading sane values (3-shunt).
 - Bus voltage measurement is correct (3S full charge is 12.6 V).
- Set conservative limits for early work:
 - Max phase current (start): **3–5 A**
 - Max mechanical speed (start): **5,000–10,000 rpm**
 - Soft ramp times: 1–3 s

Deliverable: a “safe startup profile” you can always revert to.

2) Firmware baseline with ST Motor Control SDK (Session 2–4)

Goal: get a *known-good* sensorless FOC running before you modify anything.

- Generate a project for **B-G431B-ESC1** with **sensorless speed control**.
- Make sure these work:
 - Start/Stop
 - Target speed setpoint
 - Stable estimated speed once in closed-loop

Deliverable: motor spins reliably and you can command speed from the debugger/MC tool.

3) Control pipeline understanding (Session 4–6)

Document (1 page each) with code references:

- PWM + ADC trigger timing (where currents are sampled)
- Clarke/Park
- PI current loops (I_d/I_q)
- Speed PI loop
- SVPWM
- State machine: ALIGN → OPEN-LOOP RAMP → OBSERVER CLOSED-LOOP

Deliverable: "FOC timing diagram" and "state machine diagram" for your notes.

4) Motor parameter identification & sanity checks (Session 6–8)

Sensorless performance depends heavily on parameters.

- Pole pairs: confirm (count magnets if possible).
- R_s , L_s : use Workbench/Profiler if available, or measure R_s roughly with a multimeter method + low current DC (careful).
- Use conservative observer gains initially, then tune.

Deliverable: parameter sheet (p_p , R_s , L_d/L_q or L_s , max current, max rpm).

5) Sensorless startup + handover tuning (Session 8–12)

This is where most time goes.

5.1 Alignment

- Apply I_d to lock rotor to a known electrical angle.
- Keep alignment time short (e.g., 0.2–0.5 s) and current low.

5.2 Open-loop ramp

- Ramp electrical frequency gradually while controlling I_q .
- Keep I_q limited (avoid big current spikes).
- Decide a minimum speed where observer becomes reliable (often a few thousand rpm depending on motor/load).

5.3 Handover

- Use a clear lock condition:
 - estimated speed stable
 - angle tracking error below threshold (if available)
- Blend angle from open-loop to observer smoothly (avoid torque step).

Deliverable: a startup that succeeds >90% over repeated starts at no load, then with light load.

6) Speed control tuning (Session 12–14)

- Start with slow speed loop (low bandwidth).
- Verify:
 - step response (e.g., 2k → 5k rpm)
 - no oscillation
 - current remains within limit

Deliverable: tuned speed PI values and a basic “speed step test” plot.

7) UART control layer (Session 14–16)

Do UART last so you’re not debugging control and comms at the same time.

Protocol (simple and robust)

- Commands (ASCII or binary, your choice):
 - S <rpm> set speed
 - E 1/0 enable/disable
 - Q query status
- Add safety:
 - command timeout (e.g., stop if no valid cmd for 500 ms)
 - clamp rpm (0 ... rpm_max)
 - clamp acceleration (ramp limiter)

Telemetry (very helpful)

Stream at e.g. 50–100 Hz:

- target_rpm, est_rpm, lq_ref, lq_meas, bus_voltage, fault_flags, state

Deliverable: a serial console can command speed and plot response.

Recommended initial parameter choices (so you can start today)

- PWM frequency: start around 20–30 kHz (audible reduction, decent current sampling margin).
- Speed range for early tuning: 0–10,000 rpm
- Accel limit: 1,000–3,000 rpm/s
- Current limit: start 3–5 A, increase only after stable.

Minimal experiment checklist (in order)

1. Open-loop spin test (very low current)
2. Closed-loop sensorless run at mid speed (e.g., 6k rpm)
3. Repeat start/stop 30 times (reliability)
4. Step tests in speed (2k→5k→8k rpm)

5. Add UART, repeat the same tests via commands

One thing I still can't safely assume

For the **2852 3100KV rlaarlo** motor: I'm not 100% sure of the **pole pairs** (likely 7, but verify). That affects electrical frequency and some tuning.

If you tell me **number of poles** (or pole pairs), I'll give you:

- a safe **rpm_max** to start with,
- a suggested **handover speed threshold**,
- and a concrete set of test points (rpm values + what you should observe in currents/estimation).

[https://es.aliexpress.com/i/1005010176253765.html?gatewayAdapt=glo2esp,](https://es.aliexpress.com/i/1005010176253765.html?gatewayAdapt=glo2esp)
[https://rlaarlo.com/products/riaario-2852-3100kv-engine-accessory-am-d12?srsId=AfmBOoo4YRQMzclzw0j-8TizTe6WWjcJvPRDYFeCjFILaMGrIWeuGIKG,](https://rlaarlo.com/products/riaario-2852-3100kv-engine-accessory-am-d12?srsId=AfmBOoo4YRQMzclzw0j-8TizTe6WWjcJvPRDYFeCjFILaMGrIWeuGIKG) are the same?

Short answer: **very likely yes**, but you must verify two technical details before assuming they are identical for FOC tuning.

Both links reference a:

- **Rlaarlo / Riaario 2852**
- **3100 KV**
- Brushless inrunner motor
- Typically used in 1/16–1/18 RC cars

However, in RC components (especially on AliExpress), the same model number can appear with:

- Slightly different winding batches
- Different shaft length
- Different connector type
- Sometimes different pole counts (rare, but possible)

For FOC sensorless development, only two things truly matter:

1. **Pole count (pole pairs)**
2. Approximate electrical parameters (R_s , L_s)

Most important: Pole count

A 2852 3100KV motor of this size is **almost always 2 pole pairs (4 poles)** if it is an inrunner.

That is VERY different from typical outrunners (which are 7 pole pairs or more).

If it is 2 pole pairs:

At 39,000 rpm (3S full):

- Mechanical: 39,000 rpm = 650 rps
- Electrical frequency = $650 \times 2 = 1300$ Hz

That is much easier for sensorless FOC than a 7-pole-pair outrunner.

How to confirm pole count (quick method)

1. Remove pinion (if mounted).
2. Rotate rotor slowly by hand.
3. Count magnetic detent "clicks" per mechanical revolution.
4. Divide by 2 → pole pairs.

If you count 4 strong magnetic cycles → 2 pole pairs.

If you count 14 → 7 pole pairs.

Are they the same motor?

Based on:

- Same brand (Rlaarlo / Riaario is same manufacturer line)
- Same size (28 mm diameter, 52 mm length)
- Same KV (3100 KV)
- Same typical use case

They are **almost certainly the same motor model**.

AliExpress vendors often resell the exact OEM unit.

But for control design, ignore marketing and verify:

- Pole pairs

- Shaft diameter
 - Phase wire gauge
 - Connector type
-

Good news for your FOC project

If this is a **2 pole pair inrunner**, then:

- ✓ Sensorless startup is easier
- ✓ Observer bandwidth can be lower
- ✓ Electrical frequency stays reasonable
- ✓ Current ripple is smaller

This is a very good motor choice for your first sensorless implementation.

If you confirm the pole pairs, I can:

- Compute your safe electrical frequency limits
- Suggest PWM frequency
- Suggest observer bandwidth range
- Suggest startup ramp profile tuned for this exact motor

That will save you a lot of trial-and-error.

i count 12, it could be possible?

Yes — **12 detent “clicks” per mechanical revolution is absolutely possible**, and it tells us something important.

Now let’s interpret it correctly.

What does 12 magnetic clicks mean?

When you rotate the shaft slowly and feel detents, you are feeling the interaction between:

- Rotor magnets
- Stator teeth (slots)

The number of detents per mechanical revolution is usually:

$$\text{Detents} = \frac{\text{Rotor poles} \times \text{Stator slots}}{\text{LCM relationship}}$$

So **detent count** $\neq$ **pole count directly**.

You counted:

12 detents per mechanical revolution

That is very common for small inrunner motors.

Most likely configurations

For a 2852 inrunner used in RC cars, the most common configurations are:

Option 1 (very common):

- 4 poles (2 pole pairs)
- 12 stator slots

This combination very often produces **12 detents**.

Option 2 (less common but possible):

- 6 poles (3 pole pairs)

But for 2852 3100KV, 2 pole pairs is MUCH more typical.

Let's sanity-check using KV

3100 KV motor $\rightarrow$ relatively high speed constant.

High KV motors:

- Usually fewer poles
- Lower torque per amp
- Higher base speed

If it had:

- 7 pole pairs $\rightarrow$ KV would usually be much lower.
- 2 pole pairs $\rightarrow$ KV around 3000 is very common.

So physics agrees with 2 pole pairs.

Electrical frequency calculation (assuming 2 pole pairs)

Max speed $\approx 39,000$ rpm (3S fully charged)

$$f_e = \frac{\text{RPM}}{60} \times p$$

$$f_e = \frac{39000}{60} \times 2$$

$$f_e \approx 1300 \text{ Hz}$$

That is perfectly reasonable for STM32G431.

Why 12 detents makes sense with 2 pole pairs

Most likely stator:

- 12 slots

Rotor:

- 4 poles

This is a very standard small inrunner topology.

So yes:

12 clicks is consistent with a 2 pole pair motor.

What this means for your sensorless FOC

This is actually GOOD news.

Because with 2 pole pairs:

- Electrical frequency is moderate
 - Observer tuning is easier
 - Startup ramp does not need to be extreme
 - Lower angle estimation sensitivity
-

Recommended initial control limits for your setup

Motor: 2852 3100KV

Battery: 3S

Pole pairs: 2

Start with:

- PWM frequency: 20–25 kHz
 - Max test speed (early stage): 8,000 rpm
 - Open-loop ramp target: 2,000–3,000 rpm before observer handover
 - Current limit: 4–6 A
 - Speed loop bandwidth: low (slow PI)
-

If you want, I can now:

1. Design your startup sequence (alignment + ramp + handover logic)
2. Suggest observer bandwidth values
3. Give you a week-by-week development roadmap
4. Help you compute approximate Rs/Ls ranges for tuning

You picked a very good motor to start sensorless FOC.

Design your startup sequence (alignment + ramp + handover logic)

Below is a **practical, implementable startup sequence** for **sensorless FOC speed control** on your **B-G431B-ESC1** with the **2852 3100KV motor on 3S**, assuming the motor is a **typical inrunner (likely 2 pole pairs)**. Even if pp is off by ± 1 , the structure still works; only the ramp rates and handover speed need small retuning.

0) State machine overview

IDLE → ALIGN → OPEN_LOOP_RAMP → HANDOVER_BLEND → CLOSED_LOOP → FAULT

Key signals you'll use:

- i_d, i_q measured
- i_d^*, i_q^* references

- θ_{ol} : open-loop electrical angle
 - ω_{ol} : open-loop electrical speed (rad/s)
 - $\theta_{obs}, \omega_{obs}$: observer estimates
 - V_{bus}
 - Fault flags (OCP/UV/OV/TEMP)
-

1) ALIGN (lock rotor to known electrical angle)

Purpose: make the rotor start from a known angle so open-loop ramp begins with torque in the correct direction.

What you do

- Set open-loop angle fixed:

$$\theta_{ol} = \theta_{align} \text{ (constant)}$$

- Apply **positive** i_d^* , zero i_q^* (or very small i_q^*).

Recommended parameters (start safe)

- Duration: **200–400 ms**
- i_d^* : **10–20% of your current limit** (e.g., if limit 6 A → start with 0.8–1.2 A)
- i_q^* : **0 A**
- θ_{align} : 0 rad (any fixed value is fine, but be consistent)

Safety/validation checks

- If measured current saturates or faults: abort to FAULT.
- If bus voltage low: abort.

Exit condition: alignment timer done.

2) OPEN-LOOP RAMP (spin-up without observer)

Purpose: generate back-EMF so the observer becomes reliable.

Core idea

You command the electrical angle by integrating a commanded electrical speed:

$$\theta_{ol}(k+1) = \theta_{ol}(k) + \omega_{ol}(k) T_s$$

and simultaneously command torque via i_q^* (and keep $i_d^* \approx 0$).

Structure

- Set $i_d^* = 0$ (or slightly negative if your tuning prefers, but start with 0)
- Set i_q^* to a limited value (torque command)
- Ramp electrical speed from 0 up to a target

Recommended ramp profile

Use a **linear ramp** in electrical speed (simple and robust):

$$\omega_{ol}(t) = \omega_{start} + a_\omega t$$

Where:

- ω_{start} small but nonzero to avoid stiction
- a_ω chosen so the motor accelerates smoothly

Suggested starting numbers

Pick a handover mechanical speed around **2,000–4,000 rpm** (no-load).

Assuming **2 pole pairs**:

- 3,000 rpm → 50 rps mech → 100 Hz electrical → $\omega \approx 2\pi(100) = 628$ rad/s

So set open-loop target near:

- $f_{e,target} = 80\text{--}150$ Hz (start with 100 Hz)
- Ramp duration: **0.5–1.5 s**
- i_q^* : start **10–25% current limit** (e.g., 0.6–1.5 A if 6 A limit)

Practical open-loop schedule (works well)

- First 100 ms after ALIGN: keep ω_{ol} very low (like 10–20 Hz electrical) while applying i_q^* gently
- Then ramp to 100 Hz electrical over ~1 s

Common failure modes and fixes

- **Motor squeals / doesn't accelerate**: increase i_q^* slightly or slow the ramp.
- **Current spikes / jitter**: lower i_q^* , slow ramp, check ADC timing.
- **Starts in wrong direction**: swap any two phases or invert sign convention.

Exit condition to attempt handover: reached minimum electrical speed AND observer indicates “good”.

3) Observer “good” / lock detection (what to measure)

You need a rule to decide when to trust the observer.

Use at least two of these:

1. Speed agreement

$$|\omega_{obs} - \omega_{ol}| < \Delta\omega_{lock}$$

Example: $\Delta\omega_{lock} = 10\text{--}20\%$ of ω_{ol}

2. Angle error bounded

If you can compute:

$$e_{\theta} = \text{wrap}(\theta_{obs} - \theta_{ol})$$

Require:

- $|e_{\theta}| < 20^{\circ}$ for **N consecutive samples** (e.g., 50–200 ms)

3. Observer confidence / PLL convergence

Many implementations provide an internal “converged” flag or a PLL error metric.

Lock confirmation time: require the condition to be true for **100–200 ms** to avoid false locks.

4) HANDOVER_BLEND (smooth switch from open-loop to observer)

Do not “snap” from θ_{ol} to θ_{obs} . Blend.

Blend factor

Let α go from 0 → 1 smoothly over T_{blend} :

$$\alpha(t) = \text{clamp}\left(\frac{t}{T_{blend}}, 0, 1\right)$$

Angle and speed blending

$$\theta_{cmd} = (1 - \alpha) \theta_{ol} + \alpha \theta_{obs}$$

$$\omega_{cmd} = (1 - \alpha) \omega_{ol} + \alpha \omega_{obs}$$

Use θ_{cmd} as the electrical angle for Park/InvPark while blending.

Torque continuity

Keep i_q^* continuous during blend.

Optionally reduce i_q^* slightly during blend to avoid a torque bump.

Recommended parameters

- T_{blend} : 100–300 ms

Exit condition: $\alpha = 1$ and still locked $\rightarrow$ go CLOSED_LOOP.

5) CLOSED_LOOP (speed control active)

Now:

- Angle comes purely from observer: $\theta = \theta_{obs}$
- Speed loop controls i_q^* based on speed error:

$$i_q^* = PI_\omega(\omega^* - \omega_{obs})$$

- i_d^* typically 0 (for surface PMSM) unless you're doing field weakening.

Also add:

- setpoint ramp limiter on ω^* (avoid step commands)
 - command timeout (UART safety)
-

6) Abort / fault logic (must be tight)

At any time, if:

- OCP, UV, OV, temp, or "stall" detected
- speed not increasing during ramp ("no acceleration" timeout)

then:

- disable PWM immediately
- set fault latch
- require explicit reset

No-acceleration detection example

During OPEN_LOOP_RAMP:

- if $\omega_{ol} > \omega_{min}$ and measured current is high but speed estimate stays near 0 for >300 ms $\rightarrow$ abort.

(If observer is unreliable early, you can detect "no acceleration" by monitoring BEMF proxy / observer raw amplitude if available, or simply by time + current.)

Concrete "starter values" to try (safe first test)

Assume current limit 6 A (adjust if you pick another limit).

ALIGN

- 300 ms
- $i_d^* = 1.0 \text{ A}$
- $i_q^* = 0$

OPEN LOOP

- $i_d^* = 0$
- $i_q^* = 1.0 \text{ A}$
- Ramp electrical frequency:
 - start: 15 Hz
 - target: 100 Hz
 - duration: 1.0 s

LOCK detect

- require 150 ms stable:
 - $|\omega_{obs} - \omega_{ol}| < 0.15 \omega_{ol}$
 - and $|e_\theta| < 20^\circ$ (if available)

BLEND

- 200 ms linear blend of angle source

Then enable speed PI.

UART integration points

Keep UART simple:

- During ALIGN / OPEN_LOOP / BLEND: ignore new speed commands or just store them as “pending”
- Only apply setpoint after CLOSED_LOOP
- Always allow STOP to force IDLE

If you tell me what **phase current limit** you want to start with (e.g., 5 A, 10 A) and whether you'll start **no-load (wheels off ground)**, I'll tighten the numeric values (alignment current, ramp slope, and handover frequency) to match your exact safety envelope.

Good — this screenshot is very informative.

You are running the **MCSDK Profiler**, and it is stuck at ~28% with:

“The measurement is taking too much time”
→ *Review the max speed specification*

And I immediately see the problem.

The Issue

On the right panel you set:

- Pole pairs = 2  (likely correct)
- Max speed = 10,000 RPM  (too low for this motor)
- Battery: 3S → ~12 V
- Motor: 3100 KV

Your motor is physically capable of:

$$3100 \times 12.6 \approx 39,000 \text{ RPM}$$

But you told the profiler:

“This motor can only go up to 10,000 RPM”

The profiler injects test signals based on that limit.

If the limit is too low, the estimation process becomes inconsistent and the profiler can stall.

What You Should Set

Change:

- ◆ Max speed = 35,000 RPM

(We don't use 39,000 to stay slightly conservative)

Why this matters

The profiler needs to:

- Inject current
- Accelerate the motor
- Estimate K_e and inertia

If the max speed is:

- Too low → it hits internal limit before estimation converges
- Too high → it may take long but still works

For high KV motors, the most common problem is **max speed too low**.

Important: UnderVoltage Threshold

I also see:

UnderVoltage threshold:

6 < 12 < 16

For a 3S battery:

- Full = 12.6 V
- Nominal = 11.1 V
- Safe minimum $\approx$ 9 V

Set:

- UnderVoltage = **9 V**
- OverVoltage = **13 V**

Otherwise the profiler may also behave strangely when voltage fluctuates.

Also Important: Max Current = 8 Apk

8 Apk (peak phase current) may be too low for profiling a 3100 KV motor.

Profiler often needs more dynamic current to estimate parameters.

Try:

- **Max current = 12 Apk**
- Or even 15 Apk (only during profiling)

You can reduce later.

Do This Now

1. Stop profiler
 2. Set:
 - Max speed = 35000
 - Max current = 12
 - UnderVoltage = 9
 - OverVoltage = 13
 3. Run profiling again
 4. No load (VERY important)
-

VERY IMPORTANT

Remove wheels or load.

Profiler must run with:

- Motor free spinning
- No mechanical load

Otherwise inertia estimation can fail.

Why this motor is tricky

3100 KV motors:

- Very low inductance
- Very low resistance
- Very high electrical acceleration

So profiling takes longer than typical low-KV motors.

If you want, send me:

- Battery voltage during profiling
- Whether it spins at all
- Whether you hear start attempts

And I'll fine-tune the profiler settings specifically for your motor.

the battery is 3s 2200 mah. Motor spin makes a kind of ramp process, goes to some speed and stop. The test stop with a Firmware fault detected "Over Heat"

Got it. A profiling "ramp up → stop" followed by **Firmware fault: Over Heat** usually comes from one of these (most common first):

1. **The board really is heating fast** (high current during profiling, high PWM losses, low-inductance motor).
2. **The temperature sensing is misread** (NTC not making good thermal contact, wrong sensor config, or the board reports an unrealistic high temp).
3. **The motor is not accelerating cleanly** (partial stall / wrong phase order / too aggressive ramp), so current stays high → MOSFETs heat quickly.

Here's what to do, in the order that saves the most time.

1) Confirm whether it's real overheating or a sensor reading issue

Do this first:

- Power the board, connect the tool, **do not run the motor**.
- Look at the **reported temperature** in the monitoring tool (Motor Pilot / profiler screen if available).
- Touch the board carefully (just "warm / hot" feel). If it trips "Over Heat" in seconds while the board is still cool, it's almost certainly a **temperature sensing/config** issue.

If it's truly getting hot quickly, proceed with steps 2–4.

2) Make profiling gentler (this fixes most real-overheat cases)

Your motor (3100KV) likely has **very low inductance**, so profiling can push high ripple currents.

Change these before profiling again:

Reduce stress

- **Max current:** set to **4–6 Apk** (yes, lower than before, just to get through profiling)
- **Max speed:** set high (e.g., **35,000 rpm**) so the profiler doesn't fight limits

- Keep **no-load** (wheels off, no pinion meshing anything)

Improve cooling

- Put a small **fan** blowing directly on the MOSFET side.
- Don't hold it in your hand (your hand blocks airflow).

Then retry profiling.

If it still overheats, go to step 3.

3) Check if the motor is “struggling” (causing high current)

Your description “ramps to some speed then stops” can also mean it never reaches stable estimation and draws too much current.

Quick checks:

- Swap any **two motor phases** and retry (this changes direction; sometimes it fixes a marginal startup).
- Ensure the rotor spins **perfectly freely** (no pinion contact, no tight bearings).
- Lower the “aggressiveness”:
 - Lower max current further (3–4 Apk)
 - If the profiler has “inertia / load” assumptions, set them minimal.

If it now completes profiling, you can raise limits later.

4) Fix likely temperature-threshold problems for a 3S battery setup

Some default thresholds are for other setups.

For 3S LiPo:

- **Undervoltage:** ~9.0 V (or 9.3 V conservative)
- **Overvoltage:** ~13.0 V

This doesn't directly cause overheat, but it prevents weird stops that can produce repeated restart attempts (which heat things fast).

5) If it's a false “Over Heat” (sensor/config)

If the board trips overheat while still cool:

- Verify the board's **temperature sensor** is **enabled** and **mapped correctly** in the project/profile configuration.
 - Make sure you're using the correct **B-G431B-ESC1** target profile in the ST tools.
 - If there's an option for **temperature protection threshold**, temporarily raise it slightly for testing (don't disable permanently; just diagnose).
-

Two questions I can use to pinpoint it (answer in one line each)

1. When it faults, is the board **actually hot** to the touch (yes/no)?
2. About how long from start to overheat fault (e.g., 5 s, 20 s, 60 s)?

If you can also share a screenshot of the tool's **temperature reading just before the fault**, I can tell you immediately whether it's real heating or a sensing/config problem and what exact setting to change.

maybe do you know a tentative values for the identification of the motor? R_s , L_s , I_{max} , K_e , Friction, Inertia?

Yes — I can give you **reasonable "starter" values** so you can proceed even if the profiler is failing. These will not be perfect, but they're good enough to get **sensorless FOC spinning** and then refine with measurements.

Assumptions (based on your motor type):

- 2852 **inrunner**, ~3100KV, 3S
 - Likely **2 pole pairs**
 - Low inductance, low resistance
-

1) K_e (back-EMF constant)

KV and K_e are inverses (in SI units):

$$K_e [V \cdot s/rad] \approx \frac{60}{2\pi \cdot KV}$$

For KV = 3100:

$$K_e \approx \frac{60}{2\pi \cdot 3100} \approx 0.00308 \text{ V} \cdot \text{s}/\text{rad}$$

So a good initial K_e :

- $K_e = 0.0031 \text{ V} \cdot \text{s}/\text{rad}$

If your tool wants **Vrms/kRPM** (common in ST tools), a practical initial range for a 3100KV motor is:

- $K_e \approx 2.5 \text{ to } 3.5 \text{ Vrms/kRPM}$ (start **3.0 Vrms/kRPM**)

(Exact unit conversions differ by tool definition, so use the range.)

2) R_s (phase resistance)

Typical for 28xx inrunner 3100KV:

- $R_s \text{ (phase)} \approx 0.03 \text{ to } 0.08 \Omega$

Start with:

- $R_s = 0.05 \Omega$
-

3) L_s (phase inductance)

High-KV inrunners often have very low inductance:

- $L_s \approx 5 \text{ to } 20 \mu\text{H}$ (0.005 to 0.020 mH)

Start with:

- $L_s = 0.010 \text{ mH}$ (10 μH)

If the observer is noisy or current ripple is huge, try **0.015 mH**.

4) I_{max} (phase current limit)

Your board overheated during profiling, so start conservative.

- For first sensorless bring-up: **$I_{\text{max}} = 4\text{--}6 \text{ A}$ (peak)**

Start with:

- **$I_{\text{max}} = 5 \text{ A peak}$** (then increase if stable and cool)
-

5) Friction (viscous) and Inertia

These are the least critical for basic FOC. They mainly affect speed-loop tuning and profiling.

Inertia (J)

Small inrunner rotor inertia is usually:

- $J \approx 1\text{e-}6 \text{ to } 5\text{e-}6 \text{ kg}\cdot\text{m}^2$

Start with:

- $J = 2\text{e-}6 \text{ kg}\cdot\text{m}^2$

Friction (B)

Viscous friction is typically small:

- $B \approx 1\text{e-}6 \text{ to } 1\text{e-}5 \text{ N}\cdot\text{m}\cdot\text{s}$

Start with:

- $B = 3\text{e-}6 \text{ N}\cdot\text{m}\cdot\text{s}$

If you see the speed estimate “hang” or decay too slowly, increase B slightly.

Suggested “starter table” (use this to get spinning)

- Pole pairs: 2
 - R_s : 0.05Ω
 - L_s : 0.010 mH
 - K_e : $0.0031 \text{ V}\cdot\text{s}/\text{rad}$ (or $\sim 3.0 \text{ V}_{\text{rms}}/\text{kRPM}$ if that’s the field)
 - I_{max} : 5 A peak
 - Inertia J : $2\text{e-}6 \text{ kg}\cdot\text{m}^2$
 - Friction B : $3\text{e-}6 \text{ N}\cdot\text{m}\cdot\text{s}$
-

How to refine quickly (without full profiler)

If you can do two simple measurements, you’ll improve everything:

A) R_s quick measurement (safe-ish)

- Lock rotor (don’t let it spin)
- Apply a small DC voltage between two phases through a current-limited source
- Measure current → compute line-to-line resistance → derive phase resistance

B) L_s quick estimate

Harder without an LCR meter, but you can approximate by applying a small step and measuring current slope if you have a scope + shunt reading.

Why profiling may overheat with this motor

Low R_s + very low L_s → the profiling injections can create large ripple currents and high switching losses even at moderate average current. So starting with conservative I_{max} and adding a fan helps a lot.

If you tell me **which field names/units** your ST tool is asking for (a screenshot of the "Electrical model" fields), I'll map these values precisely into the tool's exact units (especially K_e , because that one is the most confusing across tools).

 Uploaded an image

i made another test without the parameter that you shown. I tested with 35000 rpm and 6 apk, and the "over current" fault turn on

That makes sense: with a **3100KV inrunner**, the profiler's excitation can demand very fast current changes. If **Max current = 6 Apk** and it still trips **Over Current**, it usually means the profiling routine is hitting a current spike (often during alignment / injection), not that your steady-state current is 6 A.

Here's the fastest way to get past it.

1) Reduce the spike, not the average

Do these changes together:

- **Max current: 2–3 Apk** (yes, much lower just to finish identification)
- **Max speed: keep 35,000 rpm** (that part is fine)
- If there is any "profiling aggressiveness / torque / injection amplitude" setting: set it **low**.

Why: Overcurrent trips are almost always **instantaneous peaks**, and lowering Apk reduces the peak.

2) Make sure the motor is truly no-load

Even a tiny load can cause big current demand during profiling:

- Remove pinion / wheels off the ground
- Check the shaft spins freely (no rubbing, no tight bearings)

3) Check wiring and phase order (common cause of spikes)

- Ensure **solid bullet connections** (loose contact = weird current waveforms)
- Try swapping **any two phases** (this changes direction). If one wiring order causes spikes, another can behave better during alignment/ramp.

4) Power supply / battery reality check

A 3S 2200 mAh LiPo can deliver high current, so the board *can* hit overcurrent quickly. But voltage sag can also destabilize profiling and cause repeated re-tries → spikes.

- Start with a **partially charged pack** (around 11.5–12.0 V) for profiling
- Or use a bench PSU with current limit (if available)

5) Fix the thresholds for 3S

Set:

- **Undervoltage: 9.0 V**
- **Overvoltage: 13.0 V**

This won't remove overcurrent directly, but it prevents weird stop/restart cycles that create spikes.

6) If it still overcurrents at 2–3 Apk: likely current sensing timing

On low-inductance motors, if ADC sampling / PWM timing is off, the controller “sees” wrong current and reacts aggressively.

Two practical mitigations:

- Increase PWM frequency a bit (e.g., **24–30 kHz**) if your setup allows
- Ensure the project/board selection is exactly **B-G431B-ESC1** (wrong shunt/ADC config can produce fake spikes)

7) A working “safe profiling attempt” recipe

Try this exact combo:

- Pole pairs: **2**
- Max speed: **35000 rpm**
- Max current: **2.5 Apk**

- Undervoltage: 9 V
- Overvoltage: 13 V
- Motor: no-load + a small fan on the MOSFET side

If it completes, you can re-run with 3–4 Apk to improve estimates.

One key question (no need for a long answer)

When the **Over Current** fault happens, does it occur:

- **immediately at the beginning** (during alignment/injection), or
- **after it already accelerates?**

That single detail tells us whether to focus on alignment/injection limits vs ramp/handover behavior.

i was making a long ramp and after a few seconds, the over current happened

That pattern (“it ramps for a few seconds, then over-current”) usually means the profiler reaches a point where **the motor can’t follow the commanded acceleration / frequency**, so the controller keeps increasing voltage/current to maintain the ramp until it hits the current limit and trips.

For a **3100KV low-inductance inrunner**, this happens easily if any of these are true:

- ramp is too aggressive at higher speed (electrical frequency rises fast)
- supply/bus voltage sags a bit → more current needed for the same torque
- observer/injection is not locking cleanly → it “fights itself”
- max current is set too high for that test segment (spikes accumulate)

Here’s what to change to get through identification.

1) Make the ramp “two-stage” (gentle at the top)

Keep max speed high (35k), but slow the ramp especially above mid speed.

In practice:

- Stage A: 0 → 5,000 rpm (fast-ish)
- Stage B: 5,000 → 15,000 rpm (slow)

- Don't try to push toward 35k during profiling; that's just an upper bound.

If the tool doesn't allow multi-stage ramps, the workaround is:

- reduce "max speed" target for the profiling run to something like **15,000–20,000 rpm**, but keep the "max speed specification" (the limit field) high if it has separate concepts. Some ST screens only have one field, so then set it to **20,000 rpm** for the run.

2) Lower Max current during profiling (you want completion, not power)

Set **Max current** = **2–3 Apk** for profiling.

Once you have Rs/Ls/Ke, you can raise it.

Over-current during a long ramp is exactly what lowering Apk fixes.

3) Add a speed ceiling for the test (practical)

Your goal right now is parameter ID, not full-speed operation.

Set **Max speed for the test** to:

- **15,000 rpm** (first attempt)
- If it completes, try **20,000 rpm**.

With 3100KV on 3S, the motor *can* go higher, but profiling doesn't need it.

4) Avoid "field weakening" territory

At higher speeds on limited bus voltage, the controller may start pushing into regions where it needs field weakening (negative Id) to keep accelerating. During profiling, that can cause big current behavior.

So for now:

- Keep speeds moderate ($\leq 20k$)
- Keep Id reference at 0 (or whatever the tool forces)

5) Mechanical check that matters at higher speed

Even "no load" setups can become a load if something is off:

- shaft slightly bent, rubbing, bad bearing, or pinion still touching
- At high rpm, friction grows and current increases quickly.

6) Battery sag check (simple)

Even with 3S 2200 mAh, voltage can sag depending on C-rating and state of charge.
If bus voltage dips during ramp, current rises.

Do this:

- Start profiling at **11.5–12.0 V** (not fully charged 12.6 V, not low either)
- Use short wires, solid connectors

A concrete “next attempt” recipe

Try this exactly:

- Pole pairs: 2
- Max speed: **15,000 rpm**
- Max current: **2.5 Apk**
- Undervoltage: 9.0 V
- Overvoltage: 13.0 V
- Fan cooling on ESC
- Motor completely free (no pinion engagement)

If that completes, repeat at:

- 20,000 rpm, 3 Apk

Only after you have parameters, increase limits and do your full sensorless FOC tuning.

What to do if it still over-currents at 15k

Then it’s likely not “top speed demand” but a control instability in the profiling routine (current sampling / observer / phase order). In that case, the fastest discriminating test is:

- swap any two motor phases and repeat the 15k / 2.5 Apk run

If you can, tell me:

- roughly what speed it reaches when it trips (5k? 10k? 15k?)
and I’ll set a very specific ramp ceiling and current cap for your motor.

 Uploaded an image

With the last update 2APK make the test possible

Perfect — that means your main issue really was **current spikes during the ramp**, and limiting to **2 Apk** kept the profiler stable enough to finish.

Now, the values you got are a *first-pass* identification. They're usable, but we should sanity-check them (especially **Ke** and **Rs**) because the profiler can underestimate/overestimate if the run never reaches high speed or if the model definition differs.

From your screenshot I read:

- **Rs = 0.1 Ω**
- **Ls = 0.01 mH (10 μ H)**
- **Ke = 0.25 Vrms/kRPM**
- **Friction = 0.45 μ N·m·s**
- **Inertia = 0.09 μ N·m·s²**
- **Vbus = 11.6 V**
- **Max reached speed $\approx$ 16566 rpm**
- **Imax = 2 Apk**

1) Quick sanity checks (important)

Ls = 10 μ H

This is very plausible for a 3100KV inrunner. Good.

Rs = 0.1 Ω

This feels a bit high for a 2852 high-KV motor, but it can happen depending on winding and whether the tool reports "equivalent" resistance. It's not crazy, but I'd like to confirm later with a simple DC measurement.

Ke = 0.25 Vrms/kRPM

This is the one that looks suspicious **if the tool's Ke means the usual BEMF constant**:

- A 3100KV motor typically corresponds to something around **~ 3 Vrms/kRPM** (order of magnitude higher).

So either:

1. this Ke field is defined differently in your profiler (possible), or
2. the estimate is off because the run only reached ~ 16.6 k rpm and used low current, or
3. the motor KV isn't actually 3100 in reality.

Don't panic: you can still run sensorless FOC with this set, but if Ke is wrong, you'll see issues in speed scaling / observer behavior.

2) What I recommend next (minimal, high value)

Step A — Save these results as “Profile A (2 Apk)”

Keep them. This is a stable baseline.

Step B — Repeat profiling at slightly higher current (if thermals allow)

Try:

- 3 Apk, max speed 20,000 rpm
- Fan on the ESC

If it completes, compare Rs/Ls/Ke. If they change a lot, the model is not stable yet.

Step C — Do a simple Ke sanity test (fast)

You can validate KV/Ke without fancy tools:

1. Spin the motor with a drill or another motor (no ESC drive), measure phase-to-phase **AC voltage** at a known RPM (even approximate RPM).
2. Compare “volts per kRPM” roughly.

If you don’t have a tachometer, even a phone slow-motion method to estimate RPM works roughly.

This will tell us immediately if $K_e \approx 0.25 \text{ V}_{\text{rms}}/\text{kRPM}$ is realistic.

3) Use these values to design your startup sequence (initial settings)

With low inductance (10 μH) and I_{max} small, startup must be gentle:

- ALIGN: 200–300 ms, $I_d = 0.5\text{--}1.0 \text{ A}$, $I_q = 0$
- OPEN LOOP:
 - $I_q = 0.8\text{--}1.2 \text{ A}$
 - ramp to ~100 Hz electrical (for $p_p=2$ this is ~3000 rpm) over ~1 s
- HANDOVER: blend 200 ms
- CLOSED LOOP: speed PI slow at first

4) Important note about units (Friction/Inertia)

Your tool shows:

- friction in $\mu\text{N}\cdot\text{m}\cdot\text{s}$
- inertia in $\mu\text{N}\cdot\text{m}\cdot\text{s}^2$ (this is equivalent to $\text{kg}\cdot\text{m}^2$)

So:

- Inertia = $0.09 \mu\text{N}\cdot\text{m}\cdot\text{s}^2 = 0.09 \times 10^{-6} \text{ kg}\cdot\text{m}^2 = 9\text{e-}8 \text{ kg}\cdot\text{m}^2$

That’s plausible for a small inrunner rotor.

5) Next action I’d do in your place

1. Save Profile A
2. Run Profile B: 3 Apk, 20,000 rpm, fan cooling
3. Send me the screenshot of Profile B results

Then I'll tell you:

- which parameters are stable,
- which are likely wrong (especially K_e),
- and the exact startup ramp + handover thresholds tuned to your identified values.

If you prefer not to rerun profiling, tell me: **Does the motor reach 16,566 rpm smoothly, or does it "hunt" (speed oscillation) near the end?** That also indicates if K_e /observer is consistent.

 Uploaded an image

Good — this screenshot explains **exactly** what's happening.

You are not fighting the profiler anymore.

Now you are fighting **control loop scaling and speed feedback configuration**.

Let's analyze.

What I See

Mechanical speed = 17,298 RPM

Speed reference = 1,632 RPM

Yet the motor is spinning at ~17k rpm.

That means:

The speed loop is not actually regulating speed correctly.

And I see the main cause immediately:

! Speed K_p = 2515

! Speed K_i = 16

! **Kp divisor = 512**

! **Ki divisor = 256**

For a 3100KV low-inductance motor, that Kp is **extremely aggressive**.

This causes:

- Overreaction
- High torque bursts
- Current spikes
- Eventually → Over Current

Even if you command 1500 rpm, the controller likely overshoots hard and then fights itself.

Also Important

You are using:

Primary speed sensor: **STO+PLL**

Sensorless observer.

If **Ke is slightly wrong** (which is very possible),
the speed estimation scaling will be wrong.

That explains:

- Command = 1632 rpm
- Measured = 17,298 rpm

The observer speed scaling is probably off by ~10x.

That strongly suggests:

 **Your Ke value is incorrect in scaling.**

The Real Problem

Your profiler gave:

Ke = 0.25 Vrms/kRPM

But for a 3100KV motor, realistic Ke should be around:

~ 3 Vrms/kRPM

That is about 10–12x larger.

Your current value is almost exactly 10x too small.

That explains why:

- The observer thinks speed is 1.7k when it is actually ~17k.

Everything matches perfectly.

What To Do Now

Step 1 — Fix Ke manually

Change:

$K_e = 3.0 \text{ Vrms/kRPM}$

(You can fine tune later, but start here.)

Leave:

- $R_s = 0.1 \Omega$ (fine for now)
 - $L_s = 0.01 \text{ mH}$ (good)
-

Step 2 — Reduce Speed PI gains dramatically

For high KV inrunner:

Set:

Speed $K_p = 300$

Speed $K_i = 5$

Keep divisors the same.

We want a soft loop first.

Step 3 — Test at low speed

Command:

1500 rpm

Motor should now spin around that speed, not 17k.

If it still overshoots:

Reduce K_p further (200).

Why this happened

Because you limited profiling to 2 Apk and only reached ~16k rpm, the K_e identification was corrupted.

Low current → low BEMF → poor estimation → wrong scaling.

This is extremely common with high KV motors.

After Fixing K_e

Once speed scaling is correct:

1. Increase I_{max} to 4 Apk
 2. Tune speed loop gradually
 3. Only then test higher rpm
-

Important Concept

In sensorless FOC:

If K_e is wrong →

- Speed estimation is wrong
- Torque constant scaling is wrong
- Current loop interaction becomes unstable
- You get overcurrent faults

K_e is the backbone of the observer.

Next Question For You

After you set:

$K_e = 3.0$

$K_p = 300$

$K_i = 5$

Tell me:

Does the motor hold 1500 rpm smoothly?

Or does it oscillate?

That will tell us if we are fully calibrated or need small adjustments.

status fault_over "start up failure" the motor was rising
pass the 1500 rpm (set-point) and end up in 3330 rpm and
the program stop with the error

That behavior (setpoint 1500, it shoots to ~3330, then **Start up failure**) is very typical of sensorless FOC when **the startup / handover conditions are not satisfied**. In other words: it accelerates in open-loop, but the observer never "locks" cleanly, so when it tries to switch to closed-loop it fails and aborts.

We'll fix it by making startup gentler and moving the handover to a speed where the observer is reliable.

What "Start up failure" usually means in MCSDK

During **ALIGN** → **OPEN-LOOP RAMP** → **OBSERVER**, one of these happens:

- the motor doesn't follow the commanded electrical angle well enough
- the estimated speed/angle is inconsistent
- the observer-lock criteria isn't met within the allowed time
- it overshoots because torque is too high during ramp

Your overshoot to 3330 suggests **too much Iq during ramp / too aggressive ramp**, so it accelerates beyond the target before lock.

Do these changes (in this order)

1) Reduce acceleration and torque during startup

In the Workbench (or the "Rev-up" tab), change the rev-up parameters:

- **Startup Iq (or "torque during rev-up"):** reduce by 30–50%
- **Ramp duration:** increase (make it slower)
- If there are multiple rev-up phases:
 - make the later phase longer and with lower torque

Target: the motor should reach only a few thousand rpm smoothly, not jump.

If you want concrete starting numbers:

- **Iq during startup:** ~0.6–1.0 A (if your current limit is ~5–8 A)
- **Rev-up total time:** 1.5–2.5 s (slow is fine)

2) Increase the speed threshold where you hand over to the observer

Most inrunners lock well only after they generate enough back-EMF.

Set the "switch-over" / "closed-loop enable" speed to something like:

- **3000–5000 rpm** initially

Yes, even if your final command is 1500 rpm. The sequence is:

- start → lock at 3–5k → then speed loop brings it down to 1500

If the system tries to lock at 1500–2000 rpm, it often fails.

3) Add a "hold" time at the handover speed

If there is a parameter like "stabilization time":

- hold 200–400 ms at the handover speed before switching to closed loop.

This gives STO+PLL time to converge.

4) Make the speed loop even softer initially

Once it enters closed-loop, a strong speed PI can destabilize sensorless estimation.

Try:

- Speed Kp: 150–250
- Speed Ki: 2–5

Then increase later.

5) Don't try low-speed sensorless regulation first

Sensorless STO+PLL is weakest at low speed.

For first success:

- Command **5000 rpm**
- Once it runs stable for 10–20 s, then try 3000, then 1500.

This is the fastest path to a stable system.

One critical check from your screenshot

You showed earlier that the speed reading could be off by scaling (Ke uncertainty). If Ke is still wrong, startup lock will fail.

So after you adjust rev-up:

- Watch whether “mechanical speed” is believable (e.g., at 3S and no load, 5000 rpm should sound moderate, not screaming).
 - If it sounds far faster than the gauge says, Ke is still off.
-

What I need from you (just one screenshot)

Open the **Rev-up** tab and send a screenshot of the rev-up phases/parameters (the ramp table).

Then I can tell you exactly what to set: each phase duration, final speed, and Iq level so it locks reliably.

If you can't send it, tell me:

- How many rev-up phases are configured (1, 2, 3...)
- The current “final speed” of rev-up
- The current “Iq” (or torque/current) used during rev-up

 Uploaded an image

Excellent — this screenshot explains the startup failure very clearly.

Your rev-up profile is currently:

Phase 1

- 1000 ms
- Speed = 0 rpm
- Torque = 1.197 A

Phase 2

- 3333 ms
- Speed = 3330 rpm
- Torque = 1.197 A

Then flat at 3330 rpm.

This is the problem:

You are asking the motor to:

- Jump from 0 → 3330 rpm
- With constant torque
- Then immediately attempt observer lock

For a 3100KV inrunner, 3330 rpm is still relatively low BEMF.
STO+PLL may not have enough signal to lock reliably there.

That is why:

- It reaches ~3330 rpm
- Observer doesn't lock
- Startup timeout triggers
- "Start up failure" fault

The Fix (very concrete)

We will redesign your rev-up in a way that sensorless systems like.

Step 1 — Add an intermediate stage and go higher

Change to this:

Phase 1

- 500 ms
- Speed = 0 rpm
- Torque = 0.8 A

Phase 2

- 1500 ms
- Speed = 5000 rpm
- Torque = 1.0 A

Phase 3

- 1000 ms
- Speed = 6000 rpm
- Torque = 0.8 A

Remove phases 4 and 5.

What this does:

- Slow initial alignment
 - Smooth ramp
 - Reach higher speed before attempting full lock
 - Give observer time to converge
-

Why 5000–6000 rpm?

For 2 pole pairs:

6000 rpm mechanical

= 100 rps

= 200 Hz electrical

That is where STO+PLL usually becomes very stable for high-KV motors.

3330 rpm was borderline.

Step 2 — Slightly reduce torque during rev-up

1.197 A is not huge, but for low inertia it can overshoot.

Keep:

- 0.8 – 1.0 A

Less aggressive = easier lock.

Step 3 — Keep your Speed PI soft (you already fixed this)

Speed $K_p = 300$

Speed $K_i = 5$

Good. Keep that for now.

Step 4 — Test at 5000 rpm command first

Do NOT test 1500 rpm yet.

First test:

Speed reference = 5000 rpm

If it runs stable for 10–20 seconds:

Then try 3000 rpm.

Then 1500 rpm.

Sensorless always prefers medium speed first.

Important sanity check

If it still fails near 5000–6000 rpm:

Then K_e scaling is still slightly wrong.

If that happens, increase K_e slightly:

Try:

$K_e = 3.2 \text{ Vrms/kRPM}$

Small K_e changes significantly affect observer phase accuracy.

Why your current ramp fails

Graph interpretation:

Red curve = speed

Blue curve = torque

The motor reaches 3330 rpm and just sits there.

The observer likely hasn't converged fully.

When FOC switches to closed loop, angle mismatch → torque disturbance → startup failure.

After this works

We can then:

- Increase current limit to 4 Apk
 - Tune torque PI properly
 - Tune speed PI bandwidth
-

Try the new 3-phase ramp and tell me:

Does it:

- A) Lock and run stable
- B) Oscillate near 6000 rpm
- C) Fail before reaching 5000 rpm

That will tell us the next precise adjustment.

Fail before reaching 5000 rpm, Speed feedback

"Fail before reaching 5000 rpm" + **Speed feedback fault** means the firmware is deciding:

"I don't trust the speed estimate (STO+PLL), so I'm aborting."

So the motor is spinning, but the **observer speed** is not coherent enough (or is scaling wrong), and the protection triggers.

There are 3 high-probability causes. I'll give you a direct fix path that doesn't require guessing.

1) First: stop trying to regulate at low/medium speed

For STO+PLL, the easiest first win is to **lock at a higher speed**, because back-EMF is stronger and the observer becomes stable.

So change your rev-up target upward:

Rev-up proposal (more robust)

- Phase 1: 500 ms, speed 0, torque 0.8 A
- Phase 2: 1200 ms, speed 7000 rpm, torque 1.0 A
- Phase 3: 800 ms, speed 9000 rpm, torque 0.8 A

If you can only use 2 phases:

- Phase 1: 500 ms, 0 rpm, 0.8 A
- Phase 2: 2000 ms, 9000 rpm, 1.0 A

Reason: if it can't get reliable speed feedback at 5k, it often becomes fine at 8–10k.

2) Second: your K_e is still not correct

A Speed Feedback fault during rev-up is very often “observer mismatch”, and your earlier profiler K_e looked off-scale.

You should set K_e using KV (this is the best anchor you have):

For 3100 KV, a safe starting value in your tool's units is typically:

Set:

- $K_e = 3.0 \text{ Vrms/kRPM}$

Then try two nearby values if needed:

- $K_e = 2.5$
- $K_e = 3.5$

What to look for:

- If K_e is too low/high, STO+PLL angle and speed will drift → speed feedback fault.

This one parameter can make the difference between “never locks” and “locks instantly”.

3) Third: increase the *rev-up torque slightly*, but keep current limit safe

If the motor is not accelerating strongly enough, it sits in the “bad observer zone” too long and trips.

Try:

- Rev-up torque: **1.2–1.5 A**

But keep your global max current limited (since you had overcurrent before):

- **$I_{max} = 2.5\text{--}3 A_{pk}$** during tests

Also add a fan on the MOSFETs.

4) If it still fails: the speed feedback thresholds are too strict

MCSDK has thresholds like “minimum reliable speed” / “speed validation band” / “maximum speed error” (names vary by version).

What you do:

- Increase the allowed speed error band during rev-up
- Reduce the “speed feedback check” strictness
- Increase the time allowed for startup

I can tell you exactly which settings to change if you show the **Observers** tab (right side) or the Workbench sensorless observer configuration page.

Fast diagnostic that tells us if K_e is the culprit (no extra tools)

Run with:

- rev-up target = **9000 rpm**
- torque = **1.2 A**
- try $K_e = 2.5, 3.0, 3.5$ (three runs)

If one of these suddenly works, we’ve confirmed it’s primarily K_e scaling.

Quick question (one line)

When it fails, do you see the motor spinning smoothly and then it cuts, or does it “jerk / stutter” right before the fault?

- **Smooth then cut** → observer validation / K_e scaling
- **Jerk/stutter** → current sensing timing / phase wiring / unstable current loop

If you answer that + tell me your current K_e value in the project right now, I'll give you the exact next settings to try.

te motor start the ramp and stop

Thanks — “it starts the ramp and then just stops” (no stutter/jerk) points strongly to **observer validation failing** (speed feedback not trusted), not a hard current-loop instability.

So we need to get STO+PLL to lock reliably during rev-up.

1) Set K_e to a KV-based value (don't use the profiler K_e)

Your profiler K_e (0.25 Vrms/kRPM) is almost certainly wrong for a 3100KV motor.

Set:

- $K_e = 3.0$ Vrms/kRPM

Then be ready to try:

- $K_e = 2.5$
- $K_e = 3.5$

This is the single highest-leverage fix for “speed feedback” faults during sensorless start.

2) Rev-up: go higher speed, faster lock

Replace your current 3330 rpm target with this (works well for high-KV motors):

- **Phase 1:** 500 ms, Speed 0 rpm, Torque 0.8 A
- **Phase 2:** 1200 ms, Speed 7000 rpm, Torque 1.2 A
- **Phase 3:** 800 ms, Speed 9000 rpm, Torque 1.0 A

Why: STO+PLL usually becomes reliable around **8–10k rpm** on these motors (more back-EMF).

3) Keep current limit safe but not too low

- Set $I_{max} = 2.5$ Apk (you succeeded at 2 Apk; 2.5 gives a bit more margin)
- Put a fan on the MOSFETs

4) Test procedure (don't command 1500 rpm yet)

First, set Speed reference to **9000 rpm** and press Start.

If it runs stable for 10 seconds:

- then try 7000
- then 5000
- then 3000
- last: 1500

Sensorless almost always needs “prove it at medium speed first”.

5) If it still stops with Speed feedback

Then we must loosen the **speed feedback check thresholds** (MCSDK has a check like “observer speed must match open-loop speed within X% for Y ms”).

To do that, I need one screenshot:

- the **Observers** tab (right side) where STO+PLL parameters and validation thresholds are shown.

One very fast triage you can do now

Run three starts with the same rev-up (to 9000 rpm), changing only Ke:

1. $K_e = 2.5$
2. $K_e = 3.0$
3. $K_e = 3.5$

If one works, we’ve nailed the scaling. If none work, it’s threshold/config.

If you tell me what Ke value you currently have set, I’ll tell you which of the three to try first.

 Uploaded an image

target in 9000 rpm, rev-up (image) and the motor start to run with ramp and stop at 4500 rpm

Great — this is very diagnostic:

You command **target 9000 rpm**, your rev-up ramps toward 7000 then 9000, but the motor **stops around 4500 rpm**.

That usually means the **open-loop ramp is asking for more electrical speed than the motor can follow with the torque you allow**, *or* the observer validation trips right around that point and shuts it down.

We'll fix it with two changes: **give it a bit more torque earlier** and **make the speed ramp less demanding at the beginning** (so it doesn't "lose synchronism").

1) Your rev-up speed shape is too aggressive at the start

Right now Phase 2 pushes from 0 to 7000 in 1.2 s with only 1.2 A torque. For a stiff inrunner, it can fall behind and then stop.

Replace with a 3-step speed staircase (more robust)

Use lower intermediate speeds and hold briefly:

- **Phase 1:** 500 ms, Speed 0 rpm, Torque 0.9 A
- **Phase 2:** 1200 ms, Speed 3000 rpm, Torque 1.4 A
- **Phase 3:** 1200 ms, Speed 6000 rpm, Torque 1.4 A
- **Phase 4:** 800 ms, Speed 9000 rpm, Torque 1.2 A

(If you only have 3 phases available, remove phase 4 and target 6000 first.)

Why this works:

- You build back-EMF gradually
- You avoid losing sync early
- You give the observer time to converge

2) Increase torque a bit (but keep current limit safe)

Stopping at 4500 suggests torque margin is low.

- Set rev-up torque to **1.4 A** in phases 2–3 (as above)
- Keep $I_{max} = 2.5 A_{pk}$ (or 3 A_{pk} if temperatures are fine + fan)

3) Verify K_e now (because 4500 is also a classic " K_e mismatch" point)

If K_e is still wrong, STO+PLL may fail validation around mid-speed.

Do this quick test:

- Keep the new rev-up
- Try $K_e = 2.5$, then 3.0, then 3.5 $V_{rms}/kRPM$

- Do not change anything else between runs.

If one Ke value suddenly lets it pass 4500 and continue, that's the correct direction.

4) One more important check: your plot shows speed goes up then DOWN

In your rev-up plot, the red speed line peaks then decreases. That shape can confuse startup/observer switching.

For now make it strictly increasing (monotonic):

- $0 \rightarrow 3000 \rightarrow 6000 \rightarrow 9000$ (no decrease)

5) If it still stops around 4500 even with more torque

Then it's almost certainly the **speed feedback validation threshold** (observer not considered reliable yet).

In that case we adjust:

- "Minimum speed for observer validation"
- "Allowed speed error between open-loop and observer"
- "Validation time"

Those settings are in the **Observers** configuration (or Workbench sensorless parameters). If you share that screenshot, I'll tell you exactly what to relax.

Do this next (minimal steps)

1. Apply the 4-phase monotonic rev-up above
2. Try $K_e = 3.0$
3. If it still stops at ~ 4500 , try $K_e = 3.5$
4. If still stops, send the **Observers tab** screenshot

Also tell me: when it stops at 4500, does the UI show **Start up failure** or **Speed feedback fault**? That determines whether it's "lost sync" vs "observer validation."

i can't remove the 5th phase on the rev-up waht would be good setup;

No problem — if the tool forces 5 phases, we'll just make phases 4–5 "do nothing harmful" and keep the profile **monotonic** (never decreasing speed). The main goal is: **don't demand too much speed too early, and give STO+PLL time to lock.**

Here are two good 5-phase setups you can paste directly.

Setup A (recommended): gradual staircase + lock window

Use this first.

Phase 1:

- Duration: 500 ms
- Speed: 0 rpm
- Torque: 0.9 A

Phase 2:

- Duration: 900 ms
- Speed: 2500 rpm
- Torque: 1.4 A

Phase 3:

- Duration: 900 ms
- Speed: 4500 rpm
- Torque: 1.4 A

Phase 4:

- Duration: 900 ms
- Speed: 6500 rpm
- Torque: 1.2 A

Phase 5:

- Duration: 900 ms
- Speed: 9000 rpm
- Torque: 1.0 A

Why this works: it specifically targets the point where you were stopping (~4500 rpm) and adds a controlled step there.

Setup B (if it still stops near 4500): more torque earlier

If you still stop at ~4500 rpm, increase torque earlier but keep it safe.

Phase 1: 500 ms, 0 rpm, **1.0 A**

Phase 2: 900 ms, 2500 rpm, **1.6 A**

Phase 3: 900 ms, 4500 rpm, **1.6 A**

Phase 4: 900 ms, 6500 rpm, **1.3 A**

Phase 5: 900 ms, 9000 rpm, **1.1 A**

Global settings to use with both

- **Imax:** start **2.5 Apk** (fan on). If it still stalls, go to **3.0 Apk**.
 - **Ke:** start **3.0 Vrms/kRPM**. If it fails, try **3.5** next.
 - Keep your Speed PI soft (your $K_p=300$, $K_i=5$ is fine).
-

Important: what “good” looks like

After Start:

- It should pass **4500 rpm** without stopping.
 - Even if it doesn't reach 9000 immediately, it should keep accelerating smoothly.
-

One key detail (so I choose the next adjustment correctly)

When it stops, what fault lights up?

- **Start up failure** → rev-up timing/torque/lock window issue
- **Speed feedback** → Ke or observer validation thresholds issue

Reply with just the fault name you see after it stops, and your current **Ke value**, and I'll tell you the exact next tweak (either torque/time, or Ke/observer thresholds).

Speed feedback at 3036 rpm

Got it: **Speed feedback fault at ~3036 rpm.**

That is almost always **observer validation failing** (STO+PLL isn't trusted yet), not lack of torque. So the fix is:

1. make the firmware **wait longer before it starts checking speed feedback**, and/or

2. make the **handover/validation** happen at a **higher speed**, and
3. tune the observer inputs (K_e) so the estimated speed matches reality.

Below is the most effective setup you can do from your current UI constraints.

A) Rev-up 5-phase setup to avoid validation at ~3000 rpm

We will **hold around 3000 rpm** long enough for STO+PLL to lock, then go higher.

Phase 1: 500 ms, **0 rpm**, Torque **0.9 A**

Phase 2: 1200 ms, **3000 rpm**, Torque **1.4 A** ← lock window

Phase 3: 900 ms, **5000 rpm**, Torque **1.3 A**

Phase 4: 900 ms, **7000 rpm**, Torque **1.2 A**

Phase 5: 900 ms, **9000 rpm**, Torque **1.1 A**

Why: your system is failing right around 3k. This makes 3k a controlled “stabilize” step instead of blasting past it.

B) Increase the observer’s “trust” speed (the real fix)

In the **Observers** tab (right side), look for parameters like:

- *Minimum speed for observer reliability*
- *Speed validation threshold / Enable speed feedback check*
- *Observer convergence time*
- *STO PLL bandwidth / gain*
- *Speed error band* (allowed difference between open-loop and estimated speed)

Set these (rule of thumb):

- **Minimum speed for validation:** raise to **4000–5000 rpm**
- **Allowed speed error band:** loosen (e.g., from 10% → **20–30%**)
- **Validation time:** increase to **200–400 ms**

If the UI exposes only one of these, prioritize:

1. minimum validation speed
2. allowed error band

This prevents the firmware from declaring “bad speed feedback” at ~3036 rpm.

C) K_e sweep (fast and decisive)

Since speed feedback faults are extremely sensitive to K_e scaling, do this controlled test:

Keep everything the same (rev-up profile above), and run 3 trials:

1. $K_e = 2.5 \text{ Vrms/kRPM}$
2. $K_e = 3.0 \text{ Vrms/kRPM}$
3. $K_e = 3.5 \text{ Vrms/kRPM}$

If one value lets you pass beyond 3036 rpm without fault, keep it.

(You don't need perfect K_e yet — just good enough for observer lock.)

D) If you want the single best “next step” without hunting

Do this exact combination:

- Rev-up phases as in section A
- $K_e = 3.5 \text{ Vrms/kRPM}$
- $I_{\max} = 2.5 \text{ Apk}$ (fan on)

Because your fault occurs early (3k), a slightly higher K_e often stabilizes STO+PLL sooner.

What I need from you to finish this quickly

Please open the **Observers** tab and tell me what adjustable fields you see (names). Even a screenshot is fine. Then I'll tell you exactly which ones to change and to what values to eliminate the “Speed feedback at 3036 rpm” fault.

 Uploaded an image

Excellent — this screenshot is exactly what we needed.

Your issue at 3036 rpm → **Speed feedback fault** is almost certainly caused by the **B-emf consistency check being too strict for a high-KV, low-inductance motor**.

Right now you have:

- Variance threshold: **25%**
- B-emf consistency tolerance: **100%**

- B-emf consistency gain: **100%**
- Average speed depth: 64 / 64
- Sensorless speed feedback: Enabled
- Auto calculate PLL: Enabled

For a 3100KV inrunner, these defaults are often too aggressive during startup.

Why it fails at ~3000 rpm

Around 3000 rpm:

- Back-EMF is still relatively small.
- Current ripple is high (low $L_s = 10\ \mu\text{H}$).
- The observer sees noisy signals.
- The B-emf consistency test fails.
- Firmware throws **Speed feedback fault**.

So we relax the observer slightly during startup.

Change These Values

Go to the same page and change:

1 Increase variance tolerance

Change:

Variance Threshold

25% → 40%

This makes the observer less strict about speed fluctuations.

2 Reduce B-emf consistency gain

Change:

B-emf consistency gain

100% → 50%

This reduces how aggressively the firmware reacts to perceived inconsistency.

3 Reduce B-emf consistency tolerance

Change:

B-emf consistency tolerance

100% → 60%

This prevents false "bad feedback" detection at low speed.

4 Increase averaging

Change:

Average speed depth for speed loop

64 → 128

Average speed depth for observer equations

64 → 128

This smooths the noisy estimation caused by low inductance.

5 Keep Auto Calculate ON

Do not manually change G1/G2 yet.

Then Test With

Use this rev-up (monotonic, as before):

Phase 1: 500 ms, 0 rpm, 0.9 A

Phase 2: 1200 ms, 3000 rpm, 1.4 A

Phase 3: 900 ms, 5000 rpm, 1.3 A

Phase 4: 900 ms, 7000 rpm, 1.2 A

Phase 5: 900 ms, 9000 rpm, 1.1 A

$K_e = 3.0 \text{ Vrms/kRPM}$

$I_{max} = 2.5 \text{ Apk}$

What Should Happen

If this is the issue:

- It will pass 3000 rpm.
 - It may wobble slightly.
 - It should continue accelerating instead of stopping at 3036 rpm.
-

If It Still Stops at ~3000

Then we increase the lock speed threshold:

We disable speed feedback during startup and only enable it above 5000 rpm. That setting is in:

Sensorless start-up parameters tab.

Send that screenshot next if this adjustment does not fix it.

This motor is absolutely workable with STO+PLL — we just need to relax the observer because 3100KV + 10 μ H inductance is an extreme case for the default settings.

Try those four observer changes and tell me what rpm it reaches before fault.

Increase averaging does not allow 128

Understood — some MCSDK versions cap those “Average speed depth” fields at 64. No problem: we’ll fix the **speed feedback at ~3000 rpm** using only the parameters you *can* change.

Change only these (they’re enough)

1) Variance Threshold

- 25% → 45%

2) B-emf consistency tolerance

- 100% → 60%

3) B-emf consistency gain

- 100% → 50%

These three reduce false “bad speed feedback” triggers right in the 2–5k rpm range.

Keep:

- Average depths at **64**
 - Auto Calculate ON
 - Sensorless speed feedback checked (for now)
-

Keep your rev-up monotonic and add a longer “lock window” at 3000 rpm

Since the fault is at ~3036 rpm, don’t rush through that region.

Use this 5-phase rev-up:

- **P1:** 500 ms, 0 rpm, 0.9 A
- **P2:** 2000 ms, 3000 rpm, 1.4 A ← longer hold
- **P3:** 1000 ms, 5000 rpm, 1.3 A
- **P4:** 1000 ms, 7000 rpm, 1.2 A
- **P5:** 1000 ms, 9000 rpm, 1.1 A

And set:

- **I_{max} = 2.5 Apk**
 - **K_e = 3.0 V_{rms}/kRPM** (if it still fails, try 3.5 next)
-

If it STILL faults near 3000 rpm (common on high-KV)

Then the fastest workaround is to **delay speed feedback validation** until higher speed.

That setting is usually in “**Sensorless start-up parameters**” tab in the Wizard (top tabs next to Main Sensor). Look for options like:

- “Enable speed feedback after rev-up”
- “Minimum speed for closed loop”
- “Validation speed”

Set that to **5000 rpm** (or higher).

Please send a screenshot of that **Sensorless start-up parameters** tab and I’ll tell you exactly what to change (names vary by MCSDK version).

One quick check

After you apply the 3 observer changes, if it now reaches:

- > 5000 rpm before stopping → we're on the right path (fine-tune K_e and rev-up).
- still ~3000 rpm → we must adjust the "start-up parameters" validation speed.

Tell me the new RPM at which it stops after these changes.

 Uploaded an image  Uploaded an image

before the recompile with the update i find this Tab, what do you think about the info?

Perfect — this tab is **exactly where your problem is**.

And now we can explain your 3036 rpm fault precisely.

The Critical Parameter

You have:

Start-up speed threshold = 3000 rpm

That means:

As soon as open-loop reaches 3000 rpm, the firmware tries to switch to sensorless closed-loop and validate speed feedback.

But your motor is failing at **3036 rpm**.

So what's happening is:

1. Rev-up reaches ~3000 rpm
2. Firmware switches to STO+PLL
3. Observer not fully stable yet
4. Speed band tolerance fails
5. → Speed feedback fault

It's not torque.

It's not current.

It's premature switch-over.



The Real Fix

We must delay the FOC switch-over.



Change These Parameters

1 Increase Start-up speed threshold

Change:

3000 rpm → 6000 rpm

This forces open-loop to continue until 6000 rpm before trying sensorless control.

2 Increase consecutive correct measures

Change:

2 → 6

This prevents false validation during noisy transitions.

3 Relax speed band tolerance

You currently have:

Lower limit: 93.75%

Upper limit: 106.25%

That is only $\pm 6\%$.

For sensorless startup on a 3100KV motor, that is too strict.

Change to:

Lower limit: **80%**

Upper limit: **120%**

That gives $\pm 20\%$ tolerance during lock.

Increase Rev-up to FOC switch-over duration

Currently:

Duration = 25 ms

That is very short.

Change to:

100 ms

This gives smoother blending from open-loop angle to observer angle.

Why this will fix it

Right now the firmware is asking:

"At 3000 rpm, is the observer already perfect?"

Answer: No → Fault.

By raising threshold to 6000 rpm:

- Back-EMF is stronger
 - STO+PLL is stable
 - Speed validation passes
-

Your New Configuration Summary

Start-up speed threshold: **6000 rpm**

Consecutive correct measures: **6**

Speed band lower limit: **80%**

Speed band upper limit: **120%**

Rev-up to FOC switch-over duration: **100 ms**

Keep your monotonic 5-phase rev-up.

What Should Happen Now

It should:

- Pass 3000 rpm
 - Continue accelerating
 - Switch to closed-loop near 6000 rpm
 - Not stop at 3036 rpm anymore
-

Important

After it works, we can tighten tolerances again for robustness.

But first we want stable startup.

Recompile with these changes and tell me:

What rpm does it reach before fault now?

the wizard does not allow going higher than 3333 rpm?

Yes — that limit is normal in that wizard view. It's not your motor limit; it's the **wizard's rev-up target cap** for that particular startup-profile editor (it assumes "lock at low speed"). For your 3100KV motor, locking at ~3k rpm is exactly what's failing.

So we have two practical paths:

Path 1 (fastest): keep the 3333 rpm cap, but make lock easier there

If you can't raise the rev-up speed target, you must **relax the validation** at 3000–3333 rpm and give the observer more time.

In the "Start-up exit condition" section set:

1. **Start-up speed threshold:** set it to the maximum allowed (likely 3333 rpm)
2. **Consecutive correct measures:** 2 → 8

3. Speed band tolerance: widen it

- Lower limit: **70%**
- Upper limit: **130%**

4. Rev-up to FOC switch-over duration: 25 ms → 150 ms

And in the Rev-up phases (still capped at 3333):

- Keep Phase 2 speed = **3333 rpm**
- Make Phase 2 duration longer: **5000–7000 ms**
- Keep current target modest: **1.0–1.2 A**

Why this works: you're basically saying "stay at 3333 rpm long enough for STO+PLL to settle, and don't be strict while it's noisy."

This is the best you can do inside that cap.

Path 2 (correct engineering solution): unlock higher switch-over speed in Workbench

The real fix is to **raise the switch-over speed to ~6000–9000 rpm**, but that's usually not done in this limited wizard screen. You do it in the Motor Control Workbench project settings (or in the generated code parameters).

What to look for in Workbench:

- Sensorless start-up / Rev-up phase table with higher final speeds
- "Closed-loop enable speed" / "Observer enable speed" / "Rev-up end speed"
- STO/PLL validation speed

If you tell me your MCSDK version (v6.x?) and whether you're editing in **Motor Control Workbench** or only in **Motor Pilot**, I can point you to the exact page/parameter names.

What I recommend you do now (no guessing)

Do **Path 1** first to prove the motor can lock sensorless at ~3333 rpm:

- Threshold: 3333
- Correct measures: 8
- Band: 70–130%
- Switch-over duration: 150 ms
- Hold at 3333 rpm for 5–7 s

If that starts reliably, then we go to Path 2 to make it lock at a healthier speed (6–9k) and then regulate down to 1500 rpm.

One quick check

In your screenshot there is: **“Execute sensor-less algorithm starting from phase: 2”**.

Set that to **1** (if allowed).

Sometimes starting the observer earlier helps it converge before the exit condition is evaluated.

If you can, tell me:

- the maximum value you can set for **“Start-up speed threshold”** (is it also capped at 3333?), and
- whether **“Execute sensor-less algorithm starting from phase”** can be set to 1.

Then I’ll give you the exact final numbers to use.

the cap still at 3333 rpm and phase can't be change to 1
(stick to 2)

Understood. With those caps, the only way to succeed is:

- **lock STO+PLL at ~3333 rpm**, and
- **make the validation much less strict**, and
- **hold long enough at 3333** so the observer converges before the firmware judges it.

Here is a setup that usually works for high-KV motors under that constraint.

1) Rev-up profile (stay at 3333 rpm for a long time)

Keep your 5 phases like this:

- **Phase 1: Duration 1000 ms, Speed 0 rpm, Current 1.2 A**
- **Phase 2: Duration 8000 ms, Speed 3333 rpm, Current 1.2 A**
- **Phase 3: Duration 0 ms, Speed 3333 rpm, Current 1.2 A**
- **Phase 4: Duration 0 ms, Speed 3333 rpm, Current 1.2 A**
- **Phase 5: Duration 0 ms, Speed 3333 rpm, Current 1.2 A**

(If you can’t set 8000 ms, use the maximum allowed; the point is “several seconds”).

Why: your failure happens at ~3036 rpm because it tries to validate too early. This forces a long “settling time” near the validation speed.

2) Start-up exit condition (make it tolerant)

Set:

- **Start-up speed threshold: 3333 rpm (max)**
- **Consecutive correct measures: 10** (or the max you can set)
- **Speed band tolerance:**
 - Lower limit: **70%**
 - Upper limit: **130%**
- **Rev-up to FOC switch-over duration: 200 ms** (or max available)

Why:

- 2 measures is too easy to fail with noise.
- $\pm 6\%$ band is far too tight at low speed sensorless.
- 25 ms switch-over is too abrupt for STO+PLL.

3) Main Sensor page (Observer checks)

On the “Main Sensor / Observer + PLL” page set:

- **Variance threshold: 45%**
- **B-emf consistency tolerance: 60%**
- **B-emf consistency gain: 50%**

(Leave the averaging depths at 64 since you can’t increase them.)

4) Ke (must be realistic)

Set $K_e = 3.0 \text{ Vrms/kRPM}$.

If it still fails, try 3.5 next.

5) Keep current limit safe

- $I_{max} = 2.5 A_{pk}$
- Fan on the ESC

What you should observe if it’s working

- Motor ramps up and stays near 3333 rpm for several seconds
- Then it transitions to closed-loop without stopping

- After that, you can command higher speeds (e.g., 6000–9000) and then later try 1500 rpm
-

If it still gives “Speed feedback” around 3000–3300 rpm

Then STO+PLL still isn’t converging at that speed with your motor. In that case the correct fix is outside this wizard cap:

- Use a project configuration that allows higher rev-up end speed (Workbench full configuration), or
- Use a different sensorless method / settings set (some MCSDK presets), or
- Temporarily use a sensor (Hall/encoder) just to validate FOC and current loops, then come back to sensorless.

But try the settings above first — they’re the best you can do within the 3333 rpm cap.

Reply with just two things after you try it:

1. Does it reach and **hold** 3333 rpm for the whole Phase 2 duration? (yes/no)
2. What exact fault shows after it stops (Speed feedback or Start up failure)?

after those all changes the configuration does not work, the test end up with a fault_over speed Feedback. But other thing is happend, the axel of the motor is not moving, when the test start the motor sound and makes a ramp noise, but the axel never moves

Thought for a couple of seconds

If the motor makes a “ramp sound” but the shaft never turns, that’s not an observer-tuning problem anymore. That’s almost always a **power stage / wiring / commutation / mechanical lock** issue. The speed feedback fault is just the consequence of “no real rotation”.

Let’s tackle this in the right order: make sure the motor can physically spin under drive, then come back to sensorless lock.

1) Mechanical check (fast)

- Can you spin the shaft freely by hand with power OFF?
 - If it feels stiff, gritty, or locked → mechanical issue (bearing/pinion contact).
- Make sure **pinion is not engaged** with a gear. Even light contact can prevent startup.

2) Verify the motor is actually connected to the ESC phases

The “sound but no motion” symptom happens when:

- one phase is not connected (loose bullet connector)
- two phases swapped is fine; but **one phase open** gives vibration/sound without rotation

Do this:

- Check all 3 bullet connectors are tight (tug each wire).
- Inspect for cold solder joints or broken wire at the bullet.

Quick electrical sanity:

- With a multimeter (ohms), measure between phase pairs:
 - A–B, B–C, A–C should all be similar low resistance.
 - If one pair reads open/very high → that phase is disconnected.

3) Confirm it's the correct motor type selection: BLDC vs PMSM

In ST tools, selecting the wrong motor type can cause “buzzing” without rotation.

For your 2852 inrunner:

- It is typically **surface PM (SM-PMSM)** (not IPMSM).

Make sure the project/wizard motor type is **SM-PMSM**.

4) Current sensing / ADC timing mismatch (common on this board if config is wrong)

If current sensing is misconfigured, the controller can apply nonsense voltages that only produce noise.

Two checks:

- In your tool, look at measured currents (I_d/I_q). When you start, do they jump to weird values or saturate instantly?
- Ensure the selected hardware is exactly **B-G431B-ESC1** with **3-shunt** sensing (not 1-shunt).

If the board/project mismatch exists, fix that first.

5) Do a “forced open-loop spin” test (no observer)

You need a test that doesn't depend on STO+PLL:

In MCSDK / Motor Pilot, look for a mode like:

- **Voltage mode**
- **Open-loop**
- **Rev-up only**
- **FOC in open loop** (angle ramp)

Run at a low electrical frequency and small current:

- $I_q \sim 0.5\text{--}1.0\text{ A}$
- Target speed low (e.g., 500–1000 rpm equivalent)

If the shaft still never moves, that confirms it's not sensorless estimation.

6) Wiring order and direction

Even with correct wiring, sometimes initial direction sign conventions cause "push-pull" instead of rotation.

Try this:

- Swap any **two phases** and retry the open-loop spin test.

If it starts spinning after swapping, keep that order.

7) ESC/motor mismatch: inrunner + high KV at too-low current

This is less likely because you would still see *some* motion, but just in case:

- Increase rev-up current slightly (but keep I_{max} safe):
 - rev-up current 1.2 A → **1.6 A**
 - I_{max} 2.5 Apk → **3.0 Apk** (fan on)

If the motor is "stuck" by cogging and you're under-driving it, this can break it loose.

The single most important test now

Please do this and tell me the result:

Measure phase-to-phase resistance

With the motor unplugged from power (battery disconnected):

- Measure resistance A–B, B–C, A–C at the ESC motor output (or motor wires).
- Do you get three similar low readings? Or is one pair open/different?

If one phase is open, you'll get exactly what you're hearing: ramp noise, no rotation, then speed feedback fault.

If you can share one more screenshot

When you press Start and it "ramps" without moving, show the **live Id, Iq, Vbus, and fault** page. That will tell whether current is being commanded and whether current sensing is behaving.

For now, start with the multimeter phase check — it's the fastest way to confirm wiring vs configuration.

Measure phase-to-phase resistance 0.9 omh between all of them

Thought for 13s >

Good news: **all three phase-to-phase readings are equal**. That strongly suggests you do *not* have a missing phase / broken connector.

But **0.9 Ω** is **unusually high** for a 2852 3100KV *if you measured through the ESC output*. Many times the multimeter is "seeing" the ESC power stage paths (body diodes / driver circuitry), so the number isn't the true winding resistance.

Given your key symptom:

it makes the ramp sound, but the shaft never turns

That usually means one of these is true:

1. **The rotor is not actually being pulled into rotation** (too little startup torque, wrong startup mode, or the angle ramp isn't producing net torque).
2. **The power stage is not delivering real phase current** (driver not enabling, current sensing polarity mismatch, wrong hardware configuration).
3. **Mechanical lock** (less likely if you can spin by hand freely, but still check).

Below is the fastest way to isolate which one it is.

1) Measure the motor resistance “motor-only” (important)

Disconnect the motor from the ESC completely and measure at the motor wires directly:

- A–B, B–C, A–C

What you should see for a high-KV inrunner is usually **well below 1 Ω** (often a few tens to a few hundreds of milliohms; your meter may show 0.1–0.3 Ω depending on leads). If you still read ~0.9 Ω motor-only, either:

- your meter/leads dominate the reading, or
- the motor really has higher resistance than expected (still could run, but needs more voltage/current to start).

Tip: short your meter leads together and note that value, then subtract it mentally.

2) Do the “shorted phases brake test” (no tools, very telling)

With the motor disconnected:

- Spin the shaft by hand (should spin freely).
- Now **short any two motor wires together** (touch them together).
- Spin again.

Expected:

- With wires open: free spin.
- With two wires shorted: strong braking.

If you do NOT feel braking when shorted, something is wrong with the windings (unlikely, but check).

3) Confirm the ESC is producing real torque (open-loop test)

Right now your sensorless logic is failing, but even with a bad observer the **shaft should move** during rev-up if the open-loop torque is real.

In ST Motor Pilot / MCSDK, look for any mode like:

- “Open loop”
- “Voltage mode”
- “Rev-up only”
- “Force angle”
- “V/f”

If you find it, run this:

- I_q / current target: **1.5–2.0 A**
- Electrical speed: very low (equivalent to **300–800 rpm mechanical**)
- Hold 2–3 seconds

If the shaft still doesn't rotate at all, then it's not an observer issue.

If you can't find an open-loop mode, the next best test is:

Increase startup current briefly (controlled)

Because your current caps have been low (2–2.5 Apk), it's possible the motor never develops enough torque to overcome static friction/cogging.

Try:

- $I_{max} = 3.0 \text{ Apk}$
- Rev-up current targets: **1.6 A** (Phase 1 and Phase 2)
- Fan on the MOSFETs
- No load

If it starts turning, we dial it back down after.

4) Verify hardware configuration (common cause of “sound/no motion”)

If current sensing polarity or topology is wrong, FOC can “fight itself” and produce buzzing.

Double-check in Workbench:

- Board: **B-G431B-ESC1**
- Current sensing: **3-shunt**
- Motor type: **SM-PMSM**
- Pole pairs: start with 2 (we'll confirm later)

If any of these are wrong, fix them first.

5) Look at live currents during the failed start (this is decisive)

On your Motor Pilot page, when you press Start and it makes the ramp sound, watch:

- I_q (does it rise toward your target, e.g., ~1.2–1.6 A?)

- **Id**
- **Bus voltage**
- Any fault triggers before stop

Two interpretations:

- If **Iq stays near 0 A** → the driver is not actually applying torque (enable/protection/config issue).
- If **Iq rises properly but speed stays ~0** → torque is not producing rotation (wiring order / angle ramp / mechanical lock / too-low current).

What I want you to answer (two quick items)

1. Did you measure **0.9 Ω with the motor disconnected from the ESC**, or at the ESC outputs?
2. During the “ramp sound”, what do **Iq and Id** show (approx values)?

With those two answers, I can tell you exactly whether to focus on (A) boosting startup torque / changing rev-up, or (B) fixing configuration/current sensing/driver enable.

 Uploaded a file

This is the video with the operation of the motor, the axel does not move. The resistance between the cables was 0.9 omh without anything else (no esc)

Thought for 1m 14s >

Thanks — and the new symptom (“ramp sound but the shaft never turns”) is the key. At that point, **Speed feedback is just a consequence**: STO+PLL can’t estimate speed because there is **no real rotation**.

Since you measured **0.9 Ω motor-only between any pair**, wiring is likely OK (at least not open-phase). So the most likely cause is:

- the rev-up is commanding an electrical rotation that the rotor **cannot catch from standstill**, so it just “sings/buzzes” (stator field rotates, rotor doesn’t follow), and then the firmware flags Speed feedback.

This is very common when the **first commanded speed is too high**, even if the max cap is 3333 rpm.

What to do

1) Make rev-up start at a much lower speed (within your cap)

Don't jump to 3333. Use a multi-step "easy catch" ramp:

Rev-up (5 phases)

- Phase 1: **1200 ms**, Speed **0 rpm**, Current **1.6 A** (stronger alignment)
- Phase 2: **1200 ms**, Speed **500 rpm**, Current **1.6 A**
- Phase 3: **1200 ms**, Speed **1200 rpm**, Current **1.6 A**
- Phase 4: **1200 ms**, Speed **2200 rpm**, Current **1.4 A**
- Phase 5: **3000 ms**, Speed **3333 rpm**, Current **1.2 A**

Why this works: it gives the rotor time to "lock in" and follow the rotating field, instead of slipping immediately.

Keep your **I_{max}** conservative but not too low:

- **I_{max} = 3.0 Apk** (use a fan; you already saw overheat/overcurrent before)

2) Start-up exit condition (since you're forced to lock at 3333)

- Start-up speed threshold: **3333 rpm**
- Consecutive correct measures: **10**
- Speed band tolerance: **70% – 130%**
- Rev-up → FOC switch-over duration: **200 ms**

3) Set "Initial electrical angle of I_q" to 90°

In your Sensorless start-up parameters tab:

- **Initial electrical angle of I_q: 90 deg**

This often fixes "buzz but no rotation" when the initial torque-producing axis is poorly aligned.

4) Quick physical trick to confirm rotation

Put a tiny piece of tape on the shaft or a dot with a marker. Sometimes it's rotating slightly but looks still.

If the shaft still does not move at all

Then the next two most likely causes are configuration-level:

1. **Wrong pole pairs:** if pp is wrong, the "rpm" mapping is wrong and the open-loop electrical speed may be too high.
 - Try **pole pairs = 1** and repeat the test once (just as a diagnostic).
 - If it suddenly starts spinning, your pp assumption is wrong.
 2. **FOC current sensing polarity / mapping issue:** controller thinks it's injecting Iq but it isn't.
 - Watch **Iq and Id** on Motor Pilot during start:
 - If you command 1.6 A and Iq stays near 0, something is preventing torque production (enable/config/current sensing).
 - If you can share that live page (Id/Iq while buzzing), I can tell immediately.
-

The most likely fix in your case

Start with the **low-speed multi-step rev-up** (500 → 1200 → 2200 → 3333) + **Iq angle 90°**. That combination is the standard way to turn "ramp noise, no motion" into actual rotation on high-KV motors.

After you try it, tell me just this:

- Does the shaft begin to rotate even slowly during Phase 2 (500 rpm) or not?

📁 Uploaded a file 📁 Uploaded a file

now the motor moves but stop in some part (at 3330 rpm)

Thought for 25s >

Buenísimo: **ya gira**. Que se detenga justo alrededor de **3330 rpm** (que coincide con el **cap** y con el **switch-over**) casi siempre significa:

- El motor llega al final del **rev-up**,
- el firmware intenta hacer **Rev-up → FOC (Observer STO+PLL)**,
- la validación de "speed feedback" no pasa,
- y entonces corta.

O sea: ya no es "no hay torque", ahora es **handover/validación del observador en el punto 3333 rpm**.

A partir de aquí, estos son los ajustes que más suelen destrabarlo cuando estás obligado a conmutar a ~3333.

1) Haz que el Phase 5 sea un “lock hold” más largo y con un poco más de corriente

En tu rev-up, deja Phase 5 en 3333 rpm pero:

- **Phase 5 duration:** súbelo a 10,000–15,000 ms
- **Phase 5 current:** súbelo de 1.2 A a 1.4–1.6 A

Y sube el límite global para no saturarte:

- **$I_{max} = 3.0 A_{pk}$** (con ventilador)

Por qué: el observador necesita **tiempo** y un poco más de señal (BEMF + corriente) para que el PLL se “agarre” a 3333.

2) Handover más suave (esto es clave)

En “Rev-up to FOC switch-over”:

- **Duration:** ponlo en 200 ms (o el máximo que te deje)

Si lo tienes muy corto (25 ms), el cambio de ángulo/frecuencia es brusco y el PLL falla.

3) Speed band tolerance más ancho (aún más, si te deja)

Si ya lo pusiste 70–130% y todavía falla, prueba más ancho:

- **Lower:** 60%
- **Upper:** 140%

Esto es solo para que arranque. Luego se vuelve a apretar.

4) Haz un barrido de Ke (esto puede ser EL factor)

Como estás usando STO+PLL, Ke mal escalado = falla exactamente en el switch-over.

Con todo igual, prueba 3 corridas:

1. **$K_e = 2.5 V_{rms}/kRPM$**
2. **$K_e = 3.0 V_{rms}/kRPM$**
3. **$K_e = 3.5 V_{rms}/kRPM$**

Qué buscas: que en una de esas ya no se corte en 3330 y pase a control cerrado.

5) Si sigue muriendo en 3330: evita que el sistema "exija" cerrar lazo ahí

Esto es un truco cuando el wizard te encierra:

- Baja el **Speed target del Phase 5** a 3200 rpm
- Y baja **Start-up speed threshold** a 3200 rpm (si te deja)

La idea es mover el punto de conmutación lejos del borde "cap 3333" donde suele ser más frágil.

Una pregunta mínima (para elegir el siguiente paso sin adivinar)

Cuando se detiene en 3330 rpm, ¿qué fault exacto aparece?

- **Speed feedback** (lo más probable)
- o **Start up failure**

Si es Speed feedback: seguimos con **Ke + tolerancias + switch-over duration**.

Si es Start up failure: es más de **torque/rampa/tiempos**.

Si quieres, yo también puedo mirar tu video y decirte si lo que ocurre es "corte por fault" o "pierde sincronismo y se frena". Pero con lo que describes, la apuesta más fuerte es: **el handover a 3333 aún no está pasando y se arregla con Phase 5 más largo + más corriente + switch-over 200 ms + barrido de Ke**.

Start up failure

Thought for a second

Perfecto: **Start up failure** (y no "Speed feedback") significa que el sistema **no logra completar la secuencia de arranque dentro del tiempo/condiciones**, típicamente porque:

- en algún punto del rev-up el rotor **pierde sincronismo** con el ángulo eléctrico (slip), o
- la transición a FOC se intenta pero el sistema "no engancha" y se vence el tiempo, o
- el torque en el último tramo es insuficiente para sostener la velocidad objetivo.

Como te falla justo en **3330 rpm**, la solución más efectiva con tu límite de 3333 es:

1. hacer el último tramo más fácil de seguir (rampa más suave cerca de 3333), y
2. dar más torque solo en el final, y
3. dar más tiempo de “estabilización” antes de declarar fallo.

Aquí va un setup concreto que suele destrabarlo.

1) Rev-up de 5 fases (suave cerca del final)

La clave es: no “chocar” contra 3333. Llegar en pasos pequeños.

- Phase 1: 1200 ms, 0 rpm, 1.6 A
- Phase 2: 1200 ms, 500 rpm, 1.6 A
- Phase 3: 1200 ms, 1500 rpm, 1.6 A
- Phase 4: 1500 ms, 2600 rpm, 1.5 A
- Phase 5: 6000–10000 ms, 3200 rpm, 1.5–1.6 A

Sí: Phase 5 a **3200 rpm**, no 3333.

Esto evita que el algoritmo intente “pegarse” al límite exacto y reduce slip.

Si luego arranca, entonces subimos a 3333.

2) Start-up exit condition (para evitar timeout)

Con Start up failure, el “tiempo/criterio” es crítico.

- **Start-up speed threshold:** ponlo en **3200 rpm** (si te deja; si no, 3333)
 - **Consecutive correct measures:** 6 (no lo pongas demasiado alto; si es muy alto puede nunca salir)
 - **Speed band tolerance:** 70–130% (déjalo así por ahora)
 - **Rev-up to FOC switch-over duration:** 200 ms
-

3) Subir un poco el límite de corriente global

Como ya lograste que gire, necesitamos margen para no perder sincronismo al final:

- **I_{max}:** 3.0 Apk
 - Con ventilador sobre MOSFETs
-

4) Parámetro rápido que ayuda mucho: “Initial electrical angle of Iq”

- Ponlo en **90°** (si no lo has dejado ya así)

Esto mejora la probabilidad de que el torque inicial esté bien orientado.

5) Si todavía falla, el siguiente paso NO es más velocidad, es menos exigencia

Haz este cambio específico:

- Mantén Phase 5 en **3200 rpm**, pero baja corriente a **1.2 A** y alarga duración a **12–15 s**.

A veces el fallo viene por demasiado torque causando oscilación cerca del borde.

Qué me dices después de probarlo

Solo necesito una cosa para escoger el ajuste exacto:

1. ¿El motor **llega a ~3200 rpm y se queda estable unos segundos** antes de fallar, o falla justo cuando está subiendo?
- Si **falla mientras sube** → aún hay slip: subimos corriente en Phase 4–5 o hacemos Phase 4 más largo.
 - Si **llega y se sostiene** y luego falla → es el switch-over/timeout: subimos switch-over duration y bajamos consecutive measures.
-

Si quieres, también puedo mirar tu video y decirte si está “resbalando” (se oye como que intenta y se cae) o si simplemente corta por condición interna.

if a could increase Start-up speed threshold, which value should i put there?

Thought for a second

If you have the option to increase it, set the **Start-up speed threshold to 6000 rpm** as your first try.

Why 6000:

- It gives STO+PLL much stronger back-EMF than 3000–3333 rpm.
- It's still far from “high speed” for your motor (3100KV on 3S), so it's safe and usually locks reliably.

If 6000 still fails, go one step higher:

- **8000 rpm**

I would not start by going above ~10,000 rpm until everything is stable.

Practical rule:

- **6000 rpm** (default best)
- **8000 rpm** (if still not locking)
- Avoid **<5000 rpm** for first sensorless success on this high-KV motor

Once it locks reliably at 6000–8000, you can command it down to 1500 rpm in closed loop.

 Uploaded an image

Thought for a second

This looks **much better**, and you're now using the right idea: **don't try to close sensorless at 3k; close at 6k.**

That said, I see two things in your current setup that can still cause **Start up failure**:

1. **Your rev-up ends at 9000 rpm, but your start-up speed threshold is 6000 rpm.**
That's OK, but it means the switch-over may happen while you're still accelerating hard, which makes lock harder.
2. **Consecutive correct measures = 10** can be *too strict* while accelerating.
You often want fewer measures at first to let it exit startup, then you tighten later.

Here's the exact tuning I recommend from your current screen.

1) Make Phase 5 a "lock hold" around the threshold (not 9000)

Change Phase 5 speed target from **9000** → **6500 rpm** (or **6000 rpm**).

Keep Phase 5 duration **6000 ms** and current **1.6 A**.

Why: you want the observer to lock while speed is relatively steady, not while the command is ramping fast.

Recommended Rev-up table

- P1: 1200 ms, 0 rpm, 1.6 A
- P2: 1200 ms, 500 rpm, 1.6 A
- P3: 1200 ms, 1500 rpm, 1.6 A
- P4: 1500 ms, 2600 rpm, 1.6 A
- **P5: 6000 ms, 6500 rpm, 1.6 A**

(After it works reliably, you can add a later acceleration to 9000 in normal operation—not during start-up.)

2) Reduce “Consecutive correct measures”

Set:

- **10 → 4** (first test)

If it exits startup too easily but becomes unstable later, then raise it to 6.

Why: 10 correct measures at the exact moment of handover is often “never satisfied” when signals are still noisy.

3) Keep these as you have them

- Start-up speed threshold: **6000 rpm**
 - Band tolerance: **70–130%** (good for now)
 - Switch-over duration: **200 ms** (good)
 - Initial electrical angle of Iq: **90°** (good)
-

4) If it still fails: reduce startup current slightly

Your current target is flat **1.6 A** through all phases. That can be fine, but sometimes at higher speed it introduces oscillation.

If it fails near the end:

- Keep 1.6 A up to Phase 4
- Set **Phase 5 current = 1.2–1.4 A**

So:

- P5: 6500 rpm, **1.3 A**, 6000 ms
-

5) Success test sequence

Once it starts and stays in closed-loop:

- Command **6000 rpm** and hold 10 s
 - Then command **9000 rpm**
 - Then command **3000 rpm**
 - Finally try **1500 rpm**
-

If you try the two main changes (P5 speed = 6500, measures = 4) and tell me:

- at what rpm it stops (if it still stops)
- and which fault appears (Start up failure vs Speed feedback)

...I can give you the next exact adjustment (it will be either Ke tweak or Phase 5 current shaping).

 Uploaded an image

Thought for a second

These Observer + PLL settings look **reasonable and consistent** with what we wanted for your high-KV motor:

- Variance threshold **45%**  (more tolerant during startup)
- B-emf consistency tolerance **60%** 
- B-emf consistency gain **50%** 
- Averaging depths **64/64** (cap) 
- Auto-calculate ON 

So at this point, if you're still getting Start up failure, the next best leverage is **the handover conditions and the last rev-up phase**, not these observer knobs.

What I'd do next (concrete)

1) Lower "Consecutive correct measures"

If you still have it at **10**, change to:

- **4** (first test)

Then if it becomes unstable later, raise to 6.

Reason: 10 is often too strict during a ramp and can cause "startup never completes" → Start up failure.

2) Make Phase 5 a lock hold near the threshold

Don't end the rev-up at 9000 if your threshold is 6000. Hold close to the threshold:

Set Phase 5:

- Speed target: **6500 rpm** (or **6000 rpm**)
- Duration: **6000–10000 ms**
- Current: start **1.6 A**, but if it oscillates or faults near the end, reduce to **1.3–1.4 A**

3) Keep switch-over duration

- 200 ms  keep it

4) Keep the speed band tolerance

- 70–130%  keep it for now

The logic

You want the observer to lock while speed is **steady** (or slowly changing). If you try to validate while accelerating hard to 9000, the estimated speed may lag, and startup logic times out → Start up failure.

One quick, decisive check

When it fails, at what RPM does it fail now?

- If it fails **near 6000–6500** → handover/validation timing (fix with Phase 5 hold + measures=4)
- If it fails **much earlier** → torque/ramp still too aggressive or Ke scaling

If you tell me the failure RPM and fault type after applying:

- Phase 5 = 6500 rpm hold
- consecutive measures = 4

...I'll give you the next exact adjustment (either Ke sweep or current shaping in Phase 4–5).

right now i am using all the application that ST has to work in this project. I am using Workbench to configure my motor's parameters after that i generate a code, next i

open STM32CubeMX and generate again the code, and opens the project, i clean the build, rebuild and upload the code. It is possible reduce the steps between reconfiguration and deployment on hardware?

Thought for a second

Yes — you can reduce this a lot. Right now you're doing a redundant loop:

Workbench → **generate** → **CubeMX** → **generate again** → **rebuild** → **flash**

In most cases you only need **Workbench** + **one IDE build/flash**.

Here are 3 cleaner workflows (pick the one that matches how you like to work).

Option A (recommended): Workbench is the “single source of truth”

Use Workbench to generate the *final* project and avoid opening CubeMX on every change.

Flow

1. Motor Control Workbench: change parameters
2. **Generate project** (it generates code + CubeMX .ioc + project files)
3. Open the generated project in **STM32CubeIDE** (or your preferred IDE)
4. Build + flash

When you need to change parameters:

- go back to Workbench → regenerate → reopen IDE → build/flash

Key rule: do not open CubeMX unless you need to change pins/peripherals not managed by Workbench.

Why this works: Workbench already produces an .ioc and configures timers/ADCs correctly for motor control. CubeMX re-generation can overwrite things.

Option B: CubeMX only once, then never “Generate code” again

If you *must* use CubeMX (e.g., adding UART/CAN, extra pins), use it as a one-time base.

Flow

1. Workbench: configure motor → generate project once

2. Open the generated .ioc in CubeMX
3. Add only what you need (UART, CAN, GPIO, etc.)
4. Generate code **once**
5. From now on:
 - For motor parameter changes: do them in Workbench and regenerate *only the motor-control parts* (or re-import carefully), **without re-generating from CubeMX** every time.

This requires discipline because CubeMX regeneration can overwrite motor settings.

Option C: For tuning only, don't recompile at all (fastest)

If your changes are mostly:

- rev-up phases
- thresholds (speed band, consecutive measures)
- PI gains
- limits

Many of these can be adjusted live in **Motor Pilot** (MCP over UART) without reflashing, depending on how your project is configured (register exposure).

Flow

1. Flash once (a stable baseline build)
2. Do tuning in Motor Pilot
3. Only rebuild/flash when you change structural things (UART protocol, adding CAN, changing PWM freq, etc.)

This is the fastest iteration loop during commissioning.

Practical tips to cut your cycle time

1) Stop “clean build” every time

A clean rebuild is only needed if you suspect stale artifacts.

Normally:

- Build (incremental) + flash is enough.

2) Use one-click “Build & Run” in CubeIDE

Set up a single debug/run configuration so each iteration is:

- Ctrl+B → Run

3) Keep “user code” separated

If you keep custom UART/CAN code in dedicated files (not inside autogenerated regions), regeneration becomes safer and quicker.

The biggest time saver in your case

Given you’re still in bring-up/tuning:

Use **Option A + Option C**:

- Generate baseline with Workbench
 - Flash once
 - Do as much tuning as possible in Motor Pilot
 - Only regenerate/flash when needed
-

If you tell me:

- which IDE you use (CubeIDE? VS Code + make?)
- and whether you are already connected with **Motor Pilot MCP over UART**

I can propose the exact “minimal loop” for your setup (including which parameters can be changed live vs require codegen).

I am using CubeIDE, MC Workbench and Motor Pilot MCP

Thought for a second

Yes — with **CubeIDE + MC Workbench + Motor Pilot (MCP)** you can get your iteration loop down to:

- **Motor Pilot only** (no rebuild) for most tuning, and
- **Workbench → Generate → CubeIDE build/flash** only when you change structure.

Here’s a clean workflow that minimizes steps and avoids CubeMX entirely in the loop.

The minimal “fast loop” you want

Loop 1: Tuning loop (no code generation, no rebuild)

Use this for:

- Rev-up phases, startup threshold, band tolerances
- Speed/torque PI gains
- Limits (Imax, max speed), ramps
- Observer tolerances (variance, bemf consistency)
- Many sensorless parameters (depending on what's exposed)

Flow

1. Flash a stable baseline once
2. Open Motor Pilot → connect MCP over UART
3. Change parameters live
4. Start/stop tests
5. Save parameter set (Motor Pilot can export/save settings)

This should be your default loop during commissioning.

Goal: 90% of changes done without reflashing.

Loop 2: “Regenerate & flash” loop (only when needed)

Use this only when you change:

- PWM frequency / ADC triggering / current sensing topology
- Adding UART protocol parsing or CAN stack
- Changing pin assignments
- Changing which parameters are exposed to MCP
- Any change that is not runtime-tunable

Flow

1. MC Workbench: edit config
2. Generate project
3. Open in CubeIDE → Build → Flash

Important: do not open CubeMX for this loop unless you are changing peripherals/pins outside MCSDK.

How to set yourself up so Motor Pilot can tune almost everything

What you need (once) in the Workbench-generated project:

1. MCP enabled over UART
2. Parameter exposure enabled for:
 - Rev-up profile
 - Startup exit conditions
 - Observer settings (variance, bemf consistency)
 - PI gains
 - Limits

In MCSDK this is usually controlled by the “User Interface / MCP” settings and which registers are enabled.

Once this is configured, you’ll stop recompiling for most tuning.

What to stop doing right now

Don’t do: Workbench → CubeMX → generate again

That’s the biggest time sink and also a common source of “my motor control broke” because CubeMX regeneration can overwrite timing/ADC settings.

With your toolchain, **Workbench is the correct generator** for the motor-control core.

If you need to add CAN/UART custom nodes, do it in CubeIDE code, not by regenerating CubeMX every time.

A practical working structure (so regeneration doesn’t overwrite your work)

In CubeIDE:

- Put your custom UART command parser / CAN interface in separate files:
 - `app_comm_uart.c/.h`
 - `app_comm_can.c/.h`
 - `app_control_api.c/.h`
- Only call into MCSDK through a small “adapter” layer (functions like `MC_StartMotor1()` , `MC_ProgramSpeedRampMotor1()` , etc.)

Then when you regenerate from Workbench, your custom modules remain untouched.

Recommended day-to-day workflow for your exact goal (speed control + UART/CAN later)

1. Get stable sensorless start using Motor Pilot tuning only
 2. Save the working parameter set
 3. Only then implement your UART command protocol (in CubeIDE)
 4. Later add CAN (again in CubeIDE), without touching CubeMX each time
-

One question that determines how far we can go without reflashing

In Motor Pilot, can you already edit and write to the board:

- Rev-up phases
- Startup speed threshold / consecutive measures
- Observer tolerances

If yes: you can do almost everything from Motor Pilot.

If not: we enable those in the UI register map once in Workbench, then you're set.

If you share a screenshot of Motor Pilot's "Parameters" tree (what is writable), I'll tell you exactly what still requires regeneration and what you can tune live.